北方工业大学

硕 士 学 位 论 文

面向结构化数据的数据湖存取性能优化技术研究

学 生 姓 名 陈 元

学 号 2021312120124

学科(专业学位) 计算机科学与技术

研 究 方 向 大数据

导 师 赵卓峰

2024 年 6 月 4 日

中国知网 https://www.cnki.net

Research on data lake access performance optimization technology for structured data

By

CHEN Yuan

A Dissertation Submitted to

North China University of Technology

In partial fulfillment of the requirement

For the degree of

Master of Engineering

North China University of Technology

Jun, 2024

中国知网 https://www.cnki.net

面向结构化数据的数据湖存取性能优化技术研究

摘 要

随着大数据应用的深入和发展，数据湖正成为一种大数据背景下更加灵活、可扩展的数据存储和处理方案。数据湖的出现是为了解决传统数据仓库以及 Hadoop 为代表的大数据体系无法有效地存储和灵活处理大规模多样化数据的问题。数据湖架构灵活不需要提前对数据进行模式定义，支持多种数据分析和处理工具，其中数据的存取是数据湖的关键基础。同时数据湖可以存储各种类型的数据，包括结构化数据、半结构化数据和非结构化数据，而其中结构化数据仍是当前企业业务应用的核心。因此本文聚焦于数据湖在结构化数据的存取性能优化问题，包括高基数字段及复杂逻辑等问题对结构化数据写入数据湖带来的性能挑战，以及数据湖查询中扫描数据块过多等带来的查询性能挑战。

围绕数据湖应用背景下海量数据的高性能存取需求，以新型数据存储模型和分布式存储及缓存机制为目标，通过对数据湖存储结构、数据访问模式和数据处理方法进行分析，开展数据湖存取性能优化问题研究。具体工作包括：

首先对数据湖中常用的批处理和流处理这两种数据访问模式进行分析，并针对这两种访问模式中的数据分布不均和结果缓存利用率低问题，提出一种基于数据分区和缓存机制的访问优化方案，该方案结合数据湖系统中的文件系统存储方式，设计了一种基于列式存储的数据存储结构，根据不同业务场景和不用硬件情况利用哈希算法进行数据分布，使用 LRU 算法来管理缓存结果，能够提高数据访问的效率和稳定性，并通过索引优化技术和预处理方式提高数据访问速度。

其次，针对数据湖在大规模数据情况下的更新和增量计算耗时过长问题，从改进大数据 Shuffle 机制下数据倾斜角度，提出了一种频繁元素预测再分配算法。数据计算中数据块合并慢问题也是影响性能的关键因素，提出了一种分位数合并算法。根据这两种算法提出了基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案，能够提高数据处理的速度和可靠性。实验结果表明，本文提出的数据湖存取性能优化技术相对于现有方法能够有效的提高数据的存储、访问和处理效率。

最后，基于上述研究结果，重点结合 OLAP 中的商业化报表应用场景，并利用 Spark 技术、分布式文件系统，在 Hudi 数据湖开源软件基础上，本文设计并实现了一个支持存取性能优化的数据湖平台原型系统。实验结果表明，该系统满足商业化报表场景的离线写入需求、平均查询性能在 5 秒以内，QPS 达到 500。

中国知网 https://www.cnki.net

关键字：数据湖，存取性能，数据分区，并行计算，索引优化

中国知网 https://www.cnki.net

Research on data lake access performance optimization technology for structured data

Abstract

With the deepening and development of big data applications, data lake is becoming a more flexible and scalable data storage and processing scheme under the background of big data. The emergence of data lake is to solve the problem that the traditional data warehouse and the big data system represented by Hadoop cannot effectively store and flexibly deal with large-scale and diversified data. Data lake architecture is flexible and does not require schema definition of data in advance, and supports a variety of data analysis and processing tools, in which data access is the key basis of data lake. At the same time, data lakes can store various types of data, including structured data, semi-structured data and unstructured data, and structured data is still the core of current enterprise business applications. Therefore, this paper focuses on the optimization of data lake access performance of structured data, including the performance challenges caused by high cardinality fields and complex logic problems for structured data writing to the data lake, and the query performance challenges caused by too many scanned data blocks in the data lake query.

Aiming at the high performance access requirements of massive data under the background of data lake application, this paper analyzes the storage structure, data access mode and data processing method of data lake, and studies the access performance optimization problem of data lake with the goal of new data storage model and distributed storage and cache mechanism. Specific work includes:

Firstly, the two commonly used data access modes in data lake, batch processing and stream processing, were analyzed, and an access optimization scheme based on data partitioning and cache mechanism was proposed to solve the problems of uneven data distribution and low utilization of result cache in these two access modes. The scheme combined with the file system storage mode in data lake system. According to different business scenarios and without hardware, the hash algorithm is used to distribute the data, and the LRU algorithm is used to manage the cache results, which can improve the efficiency and stability of data access. The index optimization

中国知网 https://www.cnki.net

technology and preprocessing method are used to improve the speed of data access.

Secondly, aiming at the problem that the update and incremental calculation of data lake takes too long in the case of large-scale data, from the perspective of improving the data skew under the big data Shuffle mechanism, a frequent element prediction and redistribution algorithm is proposed. Slow data block merging is also a key factor affecting performance in data computation. A quantile merging algorithm is proposed. According to these two algorithms, a data processing scheme based on Spark parallel computing and HDFS distributed storage is proposed, which can improve the speed and reliability of data processing. The experimental results show that the proposed data lake access performance optimization technology can effectively improve the efficiency of data storage, access and processing compared with the existing methods.

Finally, based on the above research results, combined with the commercial report application scenario in OLAP, and using Spark technology and distributed file system, on the basis of Hudi data Lake open source software, this paper designs and implements a data lake platform prototype system supporting access performance optimization. Experimental results show that the system meets the off-line writing requirements of commercial report scenarios, the average query performance is less than 5 seconds, and the QPS reaches 500.

Key words:data lake， access performance， data partitioning， parallel computing， index optimization

中国知网 https://www.cnki.net

目　录

第 1 章 绪论.....1
1.1 研究背景与意义.....1
1.2 大数据性能优化国内外研究现状.....4
1.2.1 基于 Spark 并行处理技术性能优化.....4
1.2.2 Hadoop 性能优化相关技术.....4
1.3 本文的主要研究内容.....6
1.3.1 数据湖访问模式优化.....7
1.3.2 数据湖增量计算优化.....7
1.4 本文内容安排.....7
第 2 章 相关理论与技术概述.....9
2.1 数据湖概述.....9
2.1.1 数据湖基本概念.....9
2.1.2 数据湖发展现状.....9
2.1.3 数据湖典型技术.....10
2.2 数据湖存取关键技术.....11
2.2.1 Spark 技术.....11
2.2.2 分布式文件系统(HDFS).....14
2.3 数据存取性能优化相关研究工作.....19
2.3.1 Spark 性能优化研究.....19
2.3.2 大数据查询性能优化研究.....20
2.4 本章小结.....21
第 3 章 数据湖访问模式优化.....22
3.1 引言.....22
3.2 基于数据分区和缓存机制的访问优化方案.....23
3.2.1 基于哈希算法的数据均匀分布.....23
3.2.2 缓存机制.....25
3.2.3 索引优化与预处理技术.....28
3.3 实验与分析.....29
3.3.1 实验环境.....29
3.3.2 数据集.....29
3.3.3 实验模型构建.....29
3.3.4 基于哈希算法的数据均匀分布实验.....30
3.3.5 缓存机制实验.....32
3.3.6 索引优化与预处理实验.....32

中国知网 https://www.cnki.net

3.3.7 数据湖访问优化方案实验.....33
3.4 本章小结.....35
第 4 章 数据湖增量计算优化.....36
4.1 引言.....36
4.2 SHUFFLE 机制与数据倾斜.....37
4.2.1 Shuffle 机制.....37
4.2.2 Spark shuffle 数据倾斜.....39
4.3 小文件问题分析.....39
4.4 基于 SPARK 并行计算和 HDFS 分布式存储的数据处理方案.....40
4.4.1 频繁元素预测再分配算法(FEPD).....40
4.4.2 分位数合并算法.....42
4.5 实验与分析.....47
4.5.1 频繁元素预测再分配算法实验.....47
4.5.2 分位数合并算法实验.....47
4.5.3 数据湖增量计算优化实验.....49
4.6 本章小结.....50
第 5 章 基于 HUDI 的数据湖系统实现.....51
5.1 需求场景分析.....51
5.2 技术架构选型.....52
5.3 系统架构设计.....53
5.4 系统硬件环境与环境搭建.....54
5.5 应用场景接入.....54
5.6 性能分析与优化.....56
5.7 本章小结.....59
第 6 章 全文总结与展望.....60
6.1 全文总结.....60
6.2 后续工作展望.....60
参考文献.....61

中国知网 https://www.cnki.net

第 1 章 绪论

1.1 研究背景与意义

在互联网的初始阶段，由于各公司数据量相对较小且种类单一，其数据架构多以关系型数据库为基础，构建了相对简洁的数据管理框架。关系型数据库以其高效的数据收集、存储和分析功能，确保了数据的高品质。然而，其局限性在于所能支撑的数据量有限。随着互联网技术的迅猛发展，数据量呈现爆炸性增长，原有的基于单一关系型数据库的数据架构逐渐显现出其局限性。为了应对这一挑战，各公司开始按照业务线等维度对数据进行拆分，将不同的数据类型分别存储在独立的数据库中，如订单数据、用户数据等。这种分布式的数据存储策略虽然在一定程度上缓解了数据量过大的问题，但同时也带来了新的问题——数据孤岛现象。数据孤岛指的是在分布式的数据架构中，不同数据库中的数据相互独立，缺乏有效的互联互通机制。当业务需求涉及跨数据库的数据分析时，由于数据孤岛的存在，数据的获取和整合变得异常困难，从而极大地限制了数据的利用价值。为了解决这一问题，数据科学家和架构师们开始探索新的数据整合和分析方法，以充分发挥数据的潜在价值。

在这个背景下，数据仓库（Data Warehouse）开始崛起，其作为一种数据整合的解决方案，旨在打破数据孤岛现象。数据仓库通过集成多个数据库的数据，实现了数据的统一处理和分析，有效解决了跨数据库数据分析的难题。此外，相较于传统的关系型数据库，数据仓库在查询性能和响应速度上均有着显著的优势。然而，数据仓库的广泛应用也伴随着一系列挑战。其中，成本问题是企业普遍关注的一个关键方面。随着数据量的不断增长，数据仓库的存储和维护成本也随之上升，给企业带来了较大的经济压力。此外，采用商业版数据仓库还存在技术闭源和依赖特定企业级服务器的风险，这可能导致企业与数据仓库提供商之间形成紧密的绑定关系，进而限制了企业的自主性和灵活性。Hadoop 的问世为大数据处理领域带来了革命性的变革。Hadoop 的分布式文件系统（HDFS）和 MapReduce 计算模型，深受 Google 的 GFS（Google File System）和 MapReduce 思想的启发，并通过开源的方式广泛提供给全球用户。Hadoop 的出现成为了大数据分析领域的一个分水岭，为企业提供了一个从昂贵的闭源商业数仓转向开源免费 Hadoop 的吸引力选择。Hadoop 不仅支持多种数据类型的存储，包括文本、图片、音视频等，这些多样性是传统数据仓库所无法比拟的。Hadoop 的核心优势在于其低成本特性。对于许多企业来说，低成本往往比高性能和丰富功能更具吸引力。

中国知网 https://www.cnki.net

此外，Hadoop 还具备良好的容错性和扩展性，能够为企业不断增长的数据提供强有力的存储和计算支持。这些优势使得 Hadoop 成为了一个备受关注的大数据处理平台，为企业带来了前所未有的机遇和挑战。

随着 Hadoop 的广泛应用，其在大数据处理领域的地位日益凸显，但同时也引发了数据库专家们的诸多争议。这些专家基于多年的数据库领域经验，提出了两个重要的准则，对 Hadoop 的某些方面提出了质疑。首先，数据的一致性和结构化是数据库设计的核心。在传统数据库中，数据通常都遵循严格的 schema（模式），以确保数据的准确性和可理解性。然而，Hadoop 体系在初期并未明确引入 schema 的概念，数据的格式和结构往往隐含在代码中，或者仅存在于数据创建者的脑海中。这种缺乏明确 schema 的设计给数据的共享和使用带来了极大的挑战，使得非原数据创建者在使用数据时面临诸多障碍。不过，随着 Hive 等工具的引入，这一问题得到了显著的改善。其次，高级别的访问语言，尤其是 SQL，在数据库查询和数据分析中扮演着至关重要的角色。相比之下，Hadoop 中的 MapReduce 编程模型虽然强大，但编写 MapReduce 代码通常较为复杂，需要较高的编程技巧和经验。这使得非专业编程人员在使用 Hadoop 进行数据分析时面临较大的困难。因此，如何提供更加友好、易于使用的数据访问和分析工具，是 Hadoop 未来发展中需要重点考虑的问题。此外，尽管 Hadoop 在大数据处理领域占据重要地位，它在若干数据库核心特性上的局限性也不容忽视。具体而言，Hadoop 体系存在以下几点不足：(1) 缺乏事务支持，意味着数据写入操作在遭遇失败时，可能遗留“脏数据”，进而对下游数据消费过程引入不一致性风险；(2) 数据与架构间的不兼容性，即系统难以有效阻止不符合预定义架构的数据录入，这直接导致了数据处理阶段频繁出现异常情况；(3) 不内置对索引的支持及数据更新功能，限制了其在需要高频交互查询和数据维护场景中的应用。尽管存在上述缺陷，Hadoop 仍凭借其高容量低成本的优势，在大数据存储领域展现了显著的吸引力，有效应对了其他技术难以克服的挑战。它作为原始数据的海量存储解决方案，虽然在数据全面性上表现出色，但数据质量参差不齐。针对关键业务数据，通常采取 ETL（提取、转换、加载）流程，将其净化并迁移到数据仓库中，旨在提升查询效能与数据质量，为企业的商业智能(BI)和报表需求提供强有力支持。然而，这一实践也加剧了“数据孤岛”现象。数据在 Hadoop 与数据仓库间的重复存储不仅导致资源冗余与成本增加，还引发了数据口径不一的问题，即在不同系统间难以确定权威数据来源，影响决策的准确性和一致性。此外，此类数据架构设计加大了运维复杂度，特别是在面对 Schema 变更时，易于引发 ETL 流程的连锁故障，严重阻碍了从数据到价值转化的效率，凸显出在数据管理和分析体系中的深层次挑战。

中国知网 https://www.cnki.net

在当前数据密集型的应用环境中，用户迫切需求一种综合性能强大的系统。这种系统不仅需要拥有 Hadoop 所具备的低成本、大容量特性，用以满足数据量的高速增长与海量存储，同时还需具备数据仓库的 ACID 事务处理能力，以确保数据的一致性和可靠性。若存在一种技术方案，能够全面支持上述需求，那么它无疑将在很大程度上解决当前数据管理领域所面临的挑战。具体而言，这种系统应当能够充分利用 Hadoop 的低成本和高可扩展性优势，构建一个低成本且高效的数据存储与处理平台。同时，该系统还应具备数据仓库的事务处理能力，支持 ACID 属性，从而确保数据的完整性、一致性和可恢复性。这种综合性的技术方案，将为用户提供一个既经济又可靠的数据管理平台，助力他们在数据驱动的决策中取得更大的成功。数据湖融合了 Hadoop 和数据仓库优势的技术。正如它的定义，它是构建在低成本分布式存储之上，提供更好的事物和性能支持的统一数据存储系统。数据仓库（Data Warehouse）运作模式倾向于采用集中式架构业务人员将需求交给数据团队，通过 BI 报表工具供业务团队查询或业务分析系统展示，数据团队按照要求进行加工，开发成维度表，数据团队可以根据需要进行数据分析，开放数据给大家使用，数据团队更多的是提供工具和环境给各个业务团队使用，这些工具和环境都是由业务团队去研发和分析。

数据湖作为一种企业全量数据的统一存储系统，是当前大数据背景下为了应对数据规模的快速增长、数据多样性的增加、数据分析的需求不断增加以及低成本的存储和处理方案而产生的。数据湖的优势在于可以快速存储、处理和分析大量数据，并通过分布式、可扩展的架构支持各种企业级应用，为企业提供更好的数据管理和分析服务。

随着数据湖系统的深入应用和支撑的数据规模逐步扩大，其性能问题成为一个关键。数据湖系统的性能一方面会受到数据存储技术的影响：当前，数据湖主要采用的存储技术包括 HDFS、S3 等，数据存储技术的不同，会对数据湖的读写性能产生影响。数据湖的性能一方面会受到数据处理技术的影响：当前数据湖主要采用的数据处理技术包括 Spark、Flink 等，不同的数据处理技术也会对数据湖的数据分析、数据挖掘、机器学习等方面产生影响。针对如何进行数据湖存取性能优化就成为了当前数据湖研究的一个重要方向和挑战。为此，针对数据湖存储结构、数据湖访问模式以及数据湖增量计算三个方面，根据数据并行处理技术、数据分区、索引机制和缓存机制出发，研究数据湖存储性能。

中国知网　https://www.cnki.net

1.2 大数据性能优化国内外研究现状

1.2.1 基于 Spark 并行处理技术性能优化

Spark 是一个提供高效分布式数据处理能力的开源大数据处理框架。它提供快速的数据处理和分析能力，使用内存计算和弹性分布式数据集（RDD）。Spark 能够处理包括结构化、半结构化和非结构化数据在内的各种类型的数据。Spark 的主要特性包括六个方面：1）内存计算：内存计算技术在 Spark 中的应用，通过将数据存储在内存中，显著提升了数据处理速度。2)弹性分布式数据集(RDD)：Spark 使用 RDD 来处理数据，这是一种可靠、高效的数据集，支持数据分区和并行处理。3）多种语言支持：Spark 支持包括 Scala、Java、Python 和 R 在内的多种编程语言，使开发者使能够使用自己熟悉的编程语言进行数据处理和分析。4）统一 API：Spark 在处理不同类型的数据时，提供了让开发者使用相同代码的统一 API。5）支持多种数据源： Spark 具备多源数据集成能力，支持包括 Hadoop HDFS、Cassandra、HBase、Amazon S3 等在内的多种数据源。6）大规模数据处理：Spark 可以处理大规模数据，支持分布式计算，可以在大型集群上运行。已广泛应用于金融、医疗、电信、零售等各个领域。

Spark 技术在多元化应用场景中的普及和深化应用，其性能优化的重要性日益凸显。为了应对不断提升的性能需求挑战，学者们提出并实施了多方位的优化策略，具体涵盖应用程序设计层面的优化、系统参数的精细化配置、Shuffle 过程的改良优化以及资源的高效分配等方面。Sailfish 研究团队推出了一套卓有成效的 Shuffle 过程优化方案[3]，旨在显著提升 Spark 在数据混洗阶段的执行效率。此外，OPS 管理系统亦针对 Spark 的 Shuffle 过程展开了深入的性能调优工作，从而在原有基础上进一步增强了 Spark 的性能表现[4]。这些互补性的优化举措构成了 Spark 性能优化的整体框架，对于推动 Spark 在不同领域的应用性能提升至关重要。

1.2.2Hadoop 性能优化相关技术

Hadoop 凭借其强大的抽象能力，有效地掩盖了集群底层硬件资源地具体复杂性，赋予了应用程序开发者及用户一种直观且高效的使用方式。系统巧妙地封装了诸如复杂地流量调控机制、任务调配与资源配置策略，数据交互手段、错误恢复机制以及负载等多项技术细节。这种高层次地抽象设计不仅简化了用户操作界面，也为 Hadoop 性能优化实践铺垫了多元化的改进空间。当前针对 Hadoop 性能提升的研究领域，已明显分化为三大研究主线：第一是针对数据传输路径及其效率的优化探讨；第二是对任务调度机制与整体资源分配策略的深度定制和改进；

中国知网 https://www.cnki.net

第三则是专注于性能模型构建与系统配置参数优化的研究工作。

针对数据传输路径及其效率的优化。对 Facebook 公司维护下的 Hadoop 集群所承载的总计 127,840 个 MapReduce 作业进行全面且细致的统计性考察[7]，揭示出从 Map 阶段过渡至 Reduce 阶段的数据迁移行为在总体作业执行时长中占据显著比例，占比平均达到大约三分之一的程度。特别指出，在 Hadoop 框架内部，数据流动的核心机制在于各节点间通过网络基础设施进行的信息交换，这一环节因其对网络资源的高度倚赖，已被确认为决定 MapReduce 作业整体效能的重要瓶颈因素[8-10]。鉴于此，众多研究致力于改善 Hadoop 环境中的数据流传输效率以提升应用程序性能。在 Hadoop 生态体系里，数据的广泛传播通常借助分布式文件共享机制得以落实。具体而言，待分发的数据一旦被持久化存储于 HDFS 分布式文件系统中，所有的目标接收节点便能从中读取所需的数据内容。尽管如此，随着接收节点数量的增长，传统的单一共享文件方式往往快速暴露其局限性，表现为在大量并发读取请求下形成的传输效率瓶颈。为了应对这一挑战并优化数据广播的过程，通过利用节点之间的对等连接和协同工作，能够有效地分散数据传输的负载，提高数据的可用性和传输效率。通过将这种分布式数据分发技术应用于 Hadoop 环境，可以更有效地满足大量并发读取请求，从而显著提升数据流传输效率和应用程序性能。同时，由于位于柜内各节点间的数据传输较位于柜内各节点间的数据传输速度更快，可以采用拓扑感知式的播报方式，进一步优化数据播报流程，即当接收方提出数据请求时，数据发送优先推荐位于柜内各节点[7]。另一方面，在 MapReduce 应用操作过程中，存在着影响操作性能的倾斜现象[11]。数据倾斜包括 Map 任务执行时间的不均衡和 Reduce 任务输入数据量的差异。为了优化作业性能，需要关注并解决这些倾斜问题，例如通过优化 Map 任务的处理逻辑、调整数据集分布，以及改进分区函数等方式来减少倾斜现象的发生。

深度定制和改进任务调度机制与整体资源分配策略， Hadoop 框架为此提供了三种不同的调度机制，分别是：先进先出调度（FIFO Scheduler）、能力调度（Capacity Scheduler）以及公平调度（Fair Scheduler）[15]。允许用户根据具体的工作负载和性能需求进行灵活选择和配置，以实现最佳的任务和资源分配。

性能模型构建与系统配置参数优化是 Hadoop 性能调优的重要组成部分。Hadoop 通过抽象和封装底层细节，极大地降低了用户的使用难度。同时，Hadoop 框架提供了一系列配置参数，可以进一步提升 Hadoop 集群的运行效率和稳定性。这些配置参数的设定对于 MapReduce 应用程序的运行性能影响是非常显著的[21]。

构建性能模型与系统配置参数的优化是优化 Hadoop 性能的关键环节之一。Hadoop 通过高层次的抽象与封装手段，将底层涉及的诸多复杂组件进行了有效隐藏，进而大幅提升了用户友好度和易用性。与此同时，Hadoop 框架内嵌了一系

中国知网 https://www.cnki.net

列可配置参数，赋予用户精细调节应用底层运行特性的能力，从而实现针对性的性能优化举措。通过科学严谨的性能建模以及对系统配置参数的精准调整，有望更深层次地提升 Hadoop 集群的整体运行效能与稳定性。文献[21]强调了这一点，指出 Hadoop 中配置参数的合理设定对于显著改善 MapReduce 应用程序的性能表现具有至关重要的作用。因此，很多研究构建性能模型来找到最优参数。Hadoop 应用程序性能建模主要有两种策略。首先，一类方法基于 MapReduce 的工作流程，运用数学建模技术构建性能模型。此类模型通过对 MapReduce 运行机制进行理论分析，并结合实际使用的配置参数值，可以估算出应用程序在运行过程中的计算负荷和 I/O 需求[22]。实际上，MapReduce 应用程序的执行时间可以根据输入数据规模、集群资源状况以及计算复杂程度等因素确立函数关系，该函数可通过对历史执行数据进行线性回归等统计学方法得到[22]。此外，此类成本模型还可进一步扩展，纳入诸如 Java 堆内存占用等其他关键性能指标[23]。其次，另外一种策略则立足于统计学原理，致力于构建预测模型。在这一方面，部分研究工作首先根据作业对资源的使用特征进行分类分组，随后利用支持向量机（SVM）预测各类作业在不同配置参数下的性能表现[24]。另外一部分研究则选择了针对 MapReduce 作业运行的各个独立阶段，运用随机森林算法分别实施优化措施[24]；同时，也有研究直接应用多元机器学习技术的方式，对 MapReduce 作业整体运行过程中的性能特征进行深入剖析与量化评估[25]。

1.3 本文的主要研究内容

数据湖存储系统的性能优化研究主要集中于两个核心层面：一是数据湖的访问模式优化，二是计算密集型任务的处理效率提升。在相对简单且对即时计算需求较低的应用场景中，数据湖的主要存取活动包括数据的批量导入和实时流式写入，这两种访问模式足以满足数据的基本写入和查询需求，无需进行深度的复杂计算。然而，在涉及增量计算的场景中，数据湖的入湖过程以及查询阶段往往伴随着较为复杂的计算任务。此时，数据湖不仅要高效地接纳来自不同源头的增量数据，还需在数据写入阶段就嵌入预处理和部分计算逻辑，以减少后续查询时的计算负担。与此同时，在查询环节，为了快速响应多样化的分析需求，系统需要具备高效处理复杂聚合、联接以及其他高级分析计算的能力。因此，本文重点关注如何在不同的数据存取场景下，通过创新的数据湖访问模式设计与计算优化策略，兼顾数据入湖效率与查询性能，以应对日益增长的大数据处理需求，尤其是那些涉及增量计算的复杂业务场景。

中国知网 https://www.cnki.net

1.3.1 数据湖访问模式优化

本文提出了一种数据分区和缓存机制的访问优化方案，旨在显著改善数据湖的性能瓶颈。首先，通过合理的分区策略最大限度地减小跨节点数据交换，从而降低网络延迟，提升查询效率。其次，对于数据缓存策略，本文探讨了如何在数据湖体系结构中融入缓存组件，以针对高频访问和重复查询的数据子集进行临时存储。这一策略能够有效减少对底层存储系统的直接访问，显著加快查询响应速度，特别是在满足实时报表需求时，缓存机制的作用尤为重要。此外，索引技术在提升查询性能方面亦发挥着不可忽视的作用，在数据湖中创建索引，以快速定位和检索所需数据，从而减少全表扫描带来的性能损耗。

1.3.2 数据湖增量计算优化

数据计算效率的提升被视为关键优化路径之一，特别在处理数据湖的增量计算场景时，内存计算技术的运用显得尤为重要。内存计算机制能够显著加速数据处理速度，通过将数据直接在内存中进行计算和操作，减少了对磁盘 I/O 的依赖，这在数据文件合并过程中发挥了重要作用，因为内存计算可以快速整合和分析大量小文件，从而克服传统磁盘存储在合并操作中的性能瓶颈。同时，文件合并问题在数据湖中是性能优化的一大因素。鉴于此，本文提出了基于 Spark 并行计算框架与 HDFS 分布式存储系统的数据处理方案。Spark 以其高效的内存分布式计算能力，能够实现数据的快速加载、处理和合并，尤其是在处理海量小文件的场景下，通过其并行化计算模型，可在集群节点间平衡计算负载，高效完成文件合并任务。另一方面，HDFS 作为底层分布式存储系统，提供了大容量、高可靠性的存储服务，同时，其支持数据的分布式存储与读取，与 Spark 协同工作时，能够确保数据在各节点之间的高效流转与整合。通过优化 Spark 作业配置，不仅可以更好地处理文件合并过程中的数据分布和计算，还能够利用 HDFS 的扩展性优势，随着数据规模的增长而无缝地扩展存储与计算资源。

总体而言，本文所提出的基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案，致力于在数据湖增量计算场景下，通过内存计算技术提升数据合并效率，同时结合强大并行计算能力和可靠的分布式存储，共同解决数据文件合并问题，从而大幅度提升数据湖性能和响应速度。

1.4 本文内容安排

本文主要由 6 章内容组成，结构安排如下：

第 1 章：绪论。首先本章介绍了本文的背景及研究的目的和意义， 然后通

中国知网 https://www.cnki.net

过基于 Spark 并行处理技术优化和 Hadoop 性能优化两个方面对本文研究的内容进行了分析，并提出本文数据湖的优化方向。

第 2 章：介绍本文中的相关组件和原理，对本文接下来的研究工作扫清了障碍。

第 3 章：数据湖访问模式优化。本文通过 Spark 的 RDD 缓存机制，对数据湖中的数据进行分区缓存。由于本文采用 HDFS 分布式文件系统作为数据湖的数据存储，因此基于 HDFS 实现数据湖的索引，以提高数据查询的效率和准确性。

第 4 章：数据湖增量计算优化。目前非常流行的基于内存计算的分布式计算平台 Spark 以简单、通用、高效为特点。Spark 本质上是基于 MapReduce 原理，也承接了 MapReduce 利用散列函数对数据进行简单分割的功能。但在实际应用中，由于数据分布的不确定性，Spark 经常出现数据倾斜的问题，在 Shuffle 阶段，数据的合理分配很难通过一次分散法来实现。本文提出了频繁元素预测再分配算法，以解决数据倾斜的问题[18-20]。在 HDFS 中存放大量的小文件在不同的数据块中，会增加文件元数据的数量，从而降低读取和获取文件的效率。针对以上小文件影响读取性能的问题，提出了分位数合并算法。

第 5 章：基于 Hudi 的数据湖系统实现。本章阐述数据湖系统相关内容。

第 6 章：总结本文的工作内容，并说明哪些地方可以改进。

中国知网　https://www.cnki.net

第 2 章 相关理论与技术概述

2.1 数据湖概述

2.1.1 数据湖基本概念

在 2010 年，James Dixon 在其个人博客中首次提出了数据湖这一概念。传统数据处理场景中，大多数企业（约为 80%-90%的比例）聚焦于结构化和半结构化数据，而非非结构化数据的处理。数据来源常限定于单个应用或系统范畴，且数据属性多呈现子事务性和非事务性特点。然而，随着数据总量持续膨胀以及类型多样性增强，若一味坚持将所有数据预先整合进数据仓库，势必将导致最基础层面数据的可见度下降，而且预先整合与聚合的数据只能解答预设问题，无法应对未来可能出现的未预知数据分析需求。因此，为解决上述困境，催生了数据湖的理念。若将数据仓库比喻为经过净化、标准化和结构化处理，便于随时引用的瓶装水零售店，那么数据湖则如同一片未经雕琢的广阔水域，保持着更为原始自然的状态。数据源源不断地注入这片湖泊之中，湖泊内的各种用户均可自由地探索、深入挖掘或采样分析数据。换言之，数据湖是一个以原始形态存储数据的仓储系统，其特色在于无需预先对数据进行任何形式的结构化处理。数据湖能够容纳各种类型的数据资源。数据湖的一大核心特征是其包容性，意味着无论是结构化还是非结构化数据格式均能在其中得到存储。由于存储在数据湖中的数据在捕获时不受到预先设定的数据结构限制，故可在必要时配合“读取即分析”的模式，以促进灵活多样的数据分析活动。

2.1.2 数据湖发展现状

目前主流应用的数据湖技术主要有三种：Hudi、Iceberg、Delta Lake。国内各大厂应用较多的是 Iceberg 与 Hudi，两个技术发展各有优劣，Hudi 在变化数据捕捉（CDC）场景支持以及小文件压缩管理方面做的比较好一些，但是在与 Flink 集成以及元数据变更管理能力等方面支持的不够完善，而 Iceberg 在元数据变更等方面做的更完善，但由于机制的不同，Iceberg 在小文件压缩处理以及 CDC 场景支持的不够高效和稳定。

随着数据湖技术生态不断的发展与成熟，越来越多的公司选择了构建湖仓一体的这样一种较新的处理架构模式，可以很好的解决原有架构的一些问题，同时

中国知网 https://www.cnki.net

也带来很多新的能力，如库表 schema 高效变更，支持 mysql CDC 等能力，另外也在处理效率上有较大的提升效果，所以逐步推广应用湖仓一体会很好的帮忙业务进行强本增效。

2.1.3 数据湖典型技术

数据湖，作为一种革新的大数据基础设施，这一架构的核心目标在于推动各类计算引擎的深度融合，以灵活应对多样化的应用场景需求。为了更直观地展现数据湖架构的核心要素与运作机制，图 1 提供了一个典型的数据湖架构示意图。这一示意图不仅揭示了数据湖各组件之间的逻辑关系，还突显了数据湖在大数据处理与分析领域中的独特优势。通过数据湖的构建，可以实现对数据的全面掌控与高效利用，进而为科学研究、业务决策以及创新应用提供有力支持。

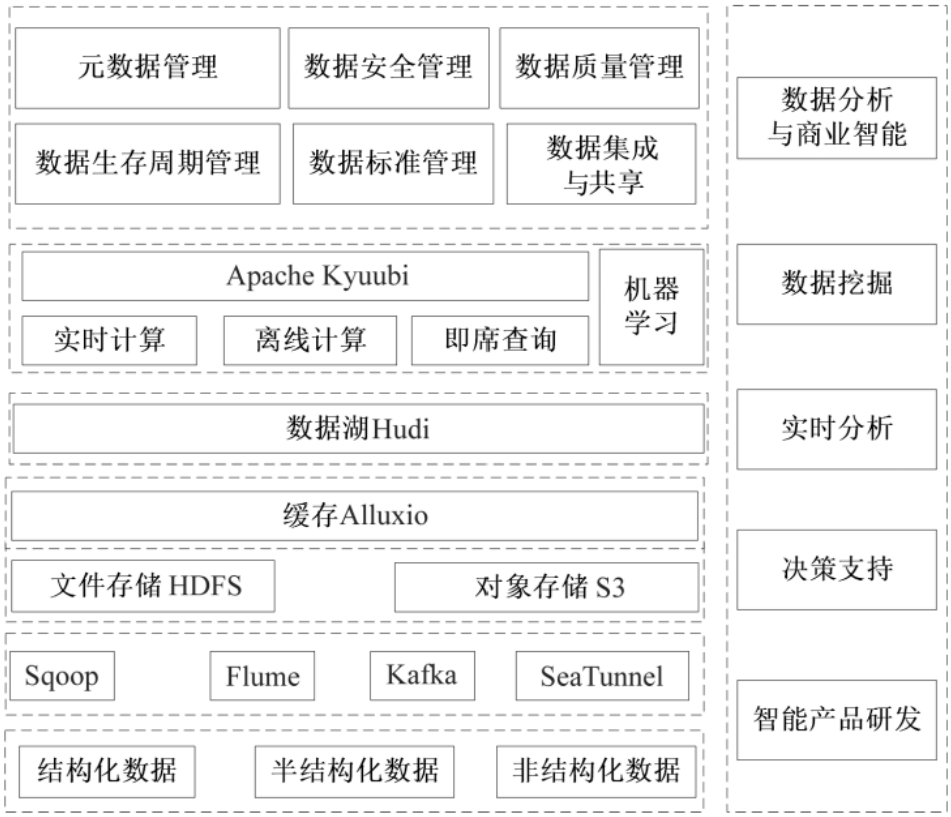

图 1 典型的数据湖架构

数据湖在存储方面具有极高的兼容性，可以按照数据原始结构进行存储，不需要进行格式转换，这是使得数据湖在机器学习场景中的应用就比较广泛。也支持在数据湖中对数据进行加工处理，用来支持报表系统的数据分析场景。数据湖存储不仅可以使用文件存储系统，也可以使用对象存储系统，这给企业带来了多种选择，对于有利于企业的将本增效。数据湖在写入方面支持离线写入和实时写

中国知网 https://www.cnki.net

入。在离线写入场景中对于 Spark 引擎的集成很友好，在实时写入场景中，也能集成 Flink 引擎。

数据湖的查询与分析算法主要包括以下三个方面：1）Hive 查询与分析，数据湖可以与 Hive SQL 集成对数据湖数据进行查询分析。2）Spark 查询与分析，数据湖与 Spark SQL 集成，可以进行查询优化。3）MapReduce 查询与分析，数据湖支持 MapReduce 框架进行查询与分析。

2.2 数据湖存取关键技术

本文中数据湖的存储采用分布式文件系统（HDFS），数据湖的查询与分析方面与 Spark 引擎集成。

2.2.1Spark 技术

Apache Spark 是大数据计算引擎。Spark 拥有 Hadoop MapReduce 所具有的优点;但不同于 MapReduce 的是 Job 中间输出结果可以保存在内存中，从而不需要进行磁盘的交互，快速处理数据能力使其非常适用于迭代算法场景中，如数据挖掘。Spark 有部分原理类似于 Hadoop MapReduce，两者之间还存较大差异，最主要的差异之处就是 Spark 采用了内存分布数据集，这些差异之处使 Spark 在有些场景下能够有更好的性能。Spark 是在 Scala 语言中实现的，它将 Scala 用作其应用程序框架。尽管 Spark 最初旨在赋能开发者高效执行分布式数据集上的迭代计算任务，实质上它扮演了 Hadoop 生态系统的有力扩展角色，能够在 Hadoop 文件系统（HDFS）环境中实现并行处理作业的无缝部署与执行。通过名为 Mesos 的第三方集群框架可以支持此行为。Spark 主要有三个特点:首先，高级 API 剥离了对集群本身的关注，Spark 应用开发者可以将重心聚焦于所要实现的计算任务本质，而非关注集群本身的复杂性。其次，Spark 在性能上很出色，不仅支持实时的交互式计算，还能够处理各种复杂的算法应用。最后，Spark 作为一个高度灵活且通用的计算引擎，能够胜任多种计算任务的执行，涵盖了多个计算领域。Spark 生态系统按功能划分为四层结构，包括：存储层、资源调度层、计算层、应用层，如图 2-1 所示。

中国知网 https://www.cnki.net

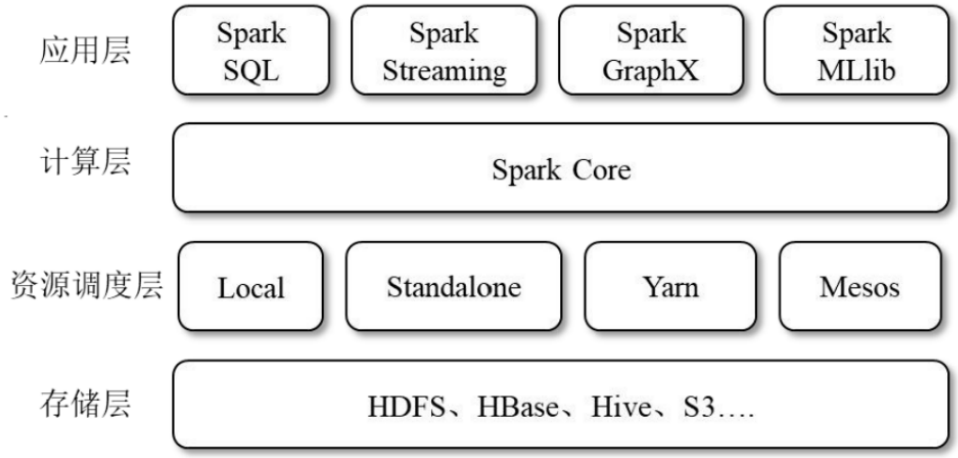

图 2-1　Spark 结构

Spark 集群有五大角色，集群管理节点 cluster manager,执行器 executor,工作节点 worker,驱动器 driver 和应用程序 application 五部分组成，如图 2-2 所示。

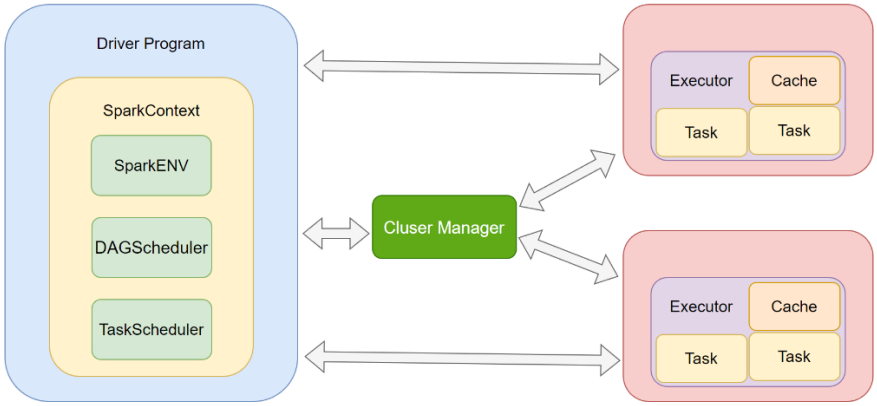

图 2-2　Spark 基本架构

下面详细说明每部分的特点。1、Cluster manager 存在于 Master 进程中，主要用来对应用程序申请的资源进行管理，根据其部署模式的不同，可以分为 local, standalone, yarn, mesos 等模式。2、worker 是 spark 的工作节点，用于执行任务的提交，主要工作职责是：1）worker 节点通过注册机向 cluster manager 汇报自身的 cpu 等信息；2)worker 节点响应 master 节点的调度指令进而初始化并激活 executor 进程， 这些 executor 实质上构成了分布式计算任务的核心执行实体。3)spark master 将任务 Task 分配给 worker 节点上的 excutor 并执行运用；4）worker 节点同步资源信息和 executor 状态信息给 cluster manager，如图 2-3 所示：

中国知网 https://www.cnki.net

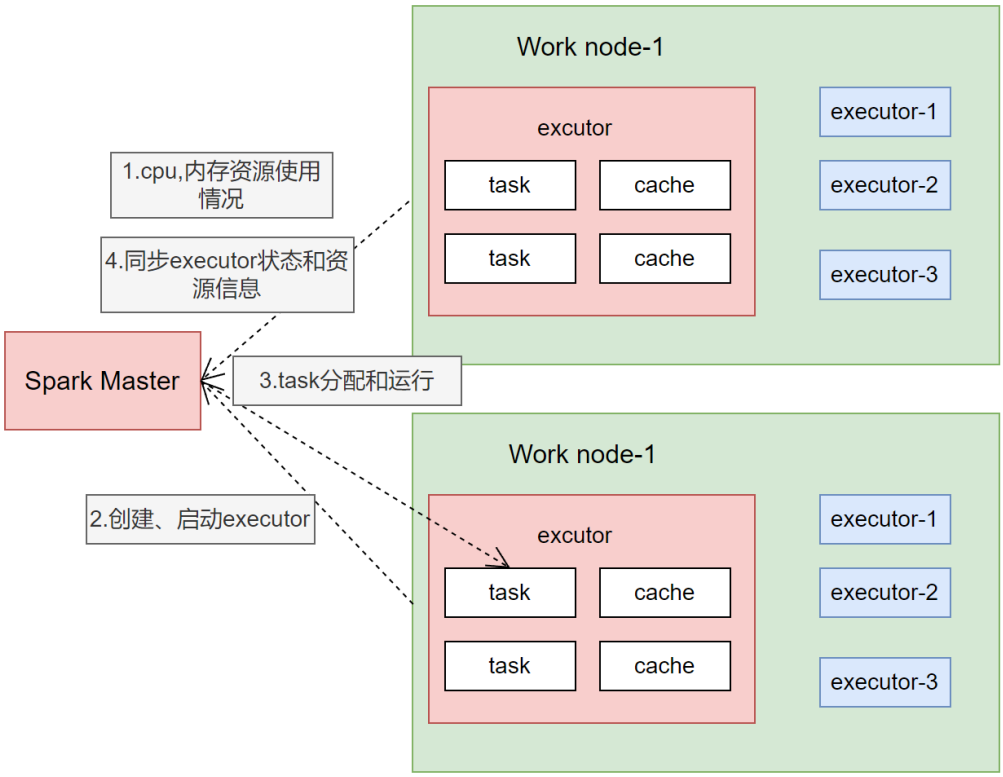

图 2-3　worker 节点职责

3、Executor 作为实际执行计算任务的核心组件，实质上体现为在 worker 节点上运行的一个特定应用程序进程，这个进程负责 Task 的运行，它能够将数据保存在内存或磁盘存储中，也能够将结构数据返回给 Driver。4、application 是 Spark API 应用程序，它包括实现 Driver 功能的代码和在程序中各个 executor 上要执行的代码，一个 application 由多个 job 组成。其中应用程序的入口为用户定义的 main 方法。5、在 Spark 体系结构中，Driver 扮演着至关重要的角色，它是指负责执行 Application 主入口函数并初始化 SparkContext 的进程实体。Application 正是通过 Driver 这一中介层与集群管理器(Cluster Manager)以及 executor 进行双向通信。Driver 进程可以根据实际需求运行在任意指定的应用程序节点之上。Driver 节点也负责提交 Job。6、SparkContext 是整个 Spark 程序最关键的一个对象，是 Spark 所有功能的主要入口点。核心作用是初始化 Spark 应用程序所需要的组件，同时还负责向 master 程序注册等。Spark 应用程序以进程为单位在分布式集群上运行，其运行过程可以细化为以下步骤：1）SparkContext 实例与集群管理器之间建立通信，申请必要的计算资源。2）cluster manager 分配应用程序执行所需要的资源，在 worker 节点创建 executor。3）SparkContext 将程序代码和 task 任务发送到 executor 上进行执行，代码可以是编译完成的 jar 包或者 python 文件等。接着 SparkContext 会收集结果到

中国知网　https://www.cnki.net

Driver 端。如图 2-4 所示：

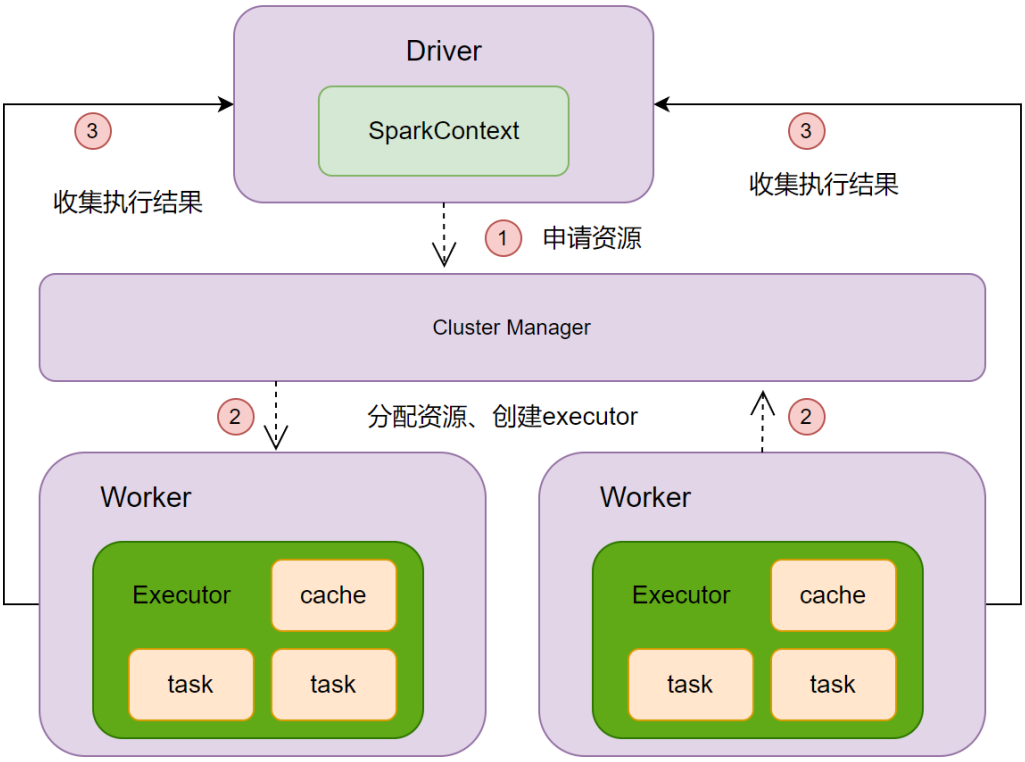

图 2-4　Spark 程序流程

2.2.2 分布式文件系统（HDFS）

在大数据情况下，一个存储系统存不下所有数据，将数据分配到多个操作系统磁盘中，为了方便管理，需要文件系统。HDFS（Hadoop Distributed File System）作为一种分布式文件系统，专为存储文件而设计，其采用目录树结构实现文件定位功能。不同于单一系统，HDFS 架构分布于众多服务器节点之上，各节点共同协作以实现系统的整体功能，且集群中不同服务器承担着各自明确的角色分工。在应用场景方面，HDFS 尤其适用于一次性写入、多次读取的场景，即文件一经创建、写入并关闭后，通常不再进行修改操作。HDFS 系统的优势主要包括：1）强大的高容错性机制：系统自动保存数据的多个副本，确保在单一副本遭受损坏或丢失时，能够通过冗余副本自动进行恢复，维持数据的完整性。2）大规模数据处理能力：HDFS 可轻松应对 GB 至 PB 级别的海量数据存储需求，并能够有效处理包含百万乃至更多数量级文件的大规模数据集。3）成本效益与可靠性兼顾：HDFS 设计初衷即为能够在低成本硬件集群上运行，通过多副本冗余策略，在保持系统高性能的同时，极大地提高了数据存储的可靠性。

HDFS 架构主要由四个关键组件构成，具体如下：1）NameNode（NN）：作为

中国知网　https://www.cnki.net

主控节点，扮演着核心管理层的角色。它负责管理和维护 HDFS 的命名空间，设定副本策略，以及管理数据块与其位置映射信息。此外，NameNode 还负责处理客户端发起的所有读写请求，确保数据访问的正确执行。2）DataNode（DN）：作为从属节点，DataNode 相当于 NameNode 的执行者，遵循其指令进行实际的数据块存储和读写操作。DataNode 是存储系统中实际承载数据块的实体，其功能不可或缺。3）Client（客户端）：客户端在与 HDFS 交互过程中起到了桥梁的作用。在文件上传过程中，Client 负责将文件切割成一个个数据块（Block），并逐个上传至 HDFS；同时，它还需与 NameNode 进行通信，获取文件的存储位置信息；并与 DataNode 进行交互，实现数据的读取或写入操作。4）Secondary NameNode（SN）：虽然并非 NameNode 的即时热备份，但在整个 HDFS 系统中扮演了重要辅助角色。Secondary NameNode 并不具备立即替代失效 NameNode 并提供服务的能力，但能有效分摊 NameNode 的部分工作负担。例如，它会定期将 Fsimage 和 Edits 文件合并，并将更新后的信息推送至 NameNode，以减轻主 NameNode 的工作压力。在紧急恢复场景下，Secondary NameNode 可协助重建 NameNode 的元数据信息。如图 2-5 所示，描绘了这些组件及其相互关系。

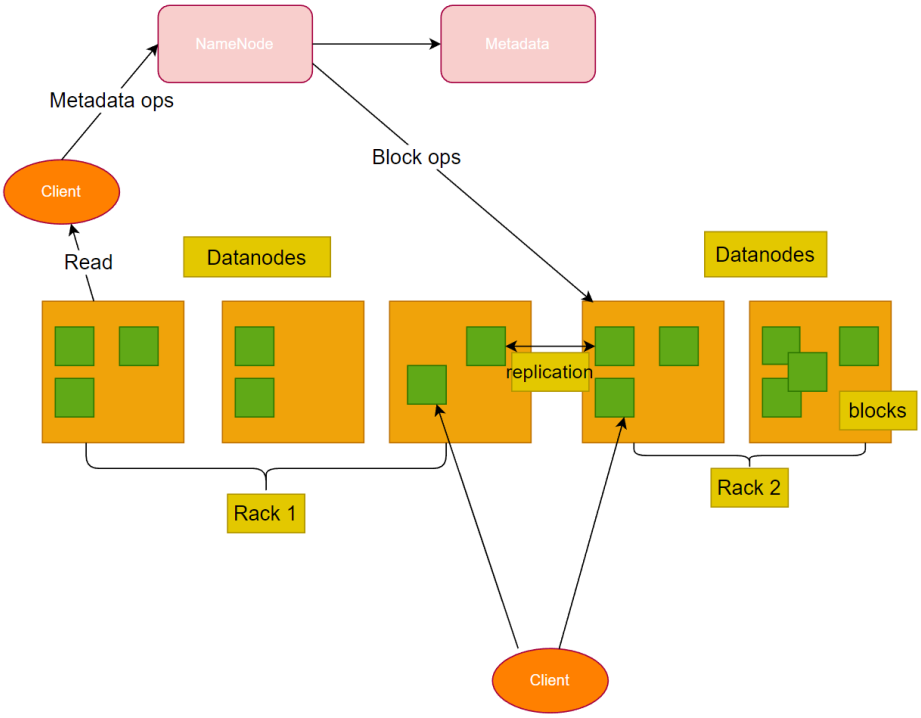

图 2-5　HDFS 架构

HDFS 默认大小在 Hadoop2.x/3.x 版本中是 128M，Hadoop1.x 版本中是 64M。HDFS 的块设置太小，会增加寻址时间，程序一直在找块的开始位置，若所设定的数据块尺寸过大，则可能出现这样的情况：从磁盘读取单个数据块所耗费的实际传输时间显著超过定位到该数据块起始点所需的寻址时间。这就意味着，在大块

中国知网　https://www.cnki.net

配置下，尽管能够减少元数据管理开销，然而，由于物理数据读取过程中的延迟增加，可能会对整体 I/O 性能造成不利影响。导致程序在处理这块数据时，会非常慢。HDFS 块的大小设置主要取决于磁盘传输速率。

HDFS 文件写入：1）客户端首先通过调用 Distributed File System (DFS) 模块向 NameNode 请求上传文件，并检查是否可以上传。2）在完成上述验证步骤后，NameNode 会依据核查结果向客户端回复上传操作是否可行的通知。3）客户端请求第一个数据块上传到服务器的具体位置。4)NameNode 返回 3 个数据节点，分别为数据节点 1，数据节点 2，数据节点 3。5) 客户端通过调用 FSDataOutputStream 模块启动数据上传流程，向首个 DataNode（DataNode1）提交上传请求。DataNode1 接收到该请求后，会进一步向下游 DataNode(DataNode2) 发出通信请求，继而 DataNode2 再将这一流程延伸至下一个 DataNode（DataNode3），如此层层传递，最终构建起一条完整的数据通信通路。6)在通信管道构建完成后，DataNode1、DataNode2 和 DataNode3 会依次响应客户端，确认它们已准备就绪，可以接受并处理数据流。这一过程体现了逐级响应机制，确保客户端能够及时了解到整个数据传输通道的建立情况。7）客户端先从磁盘读取数据放到一个本地内存缓存，以 Packet 为单位，接下来就会根据副本规则进行副本同步到其他节点。8）当一个数据块传输完成之后，客户端会再次请求 NameNode 上传。写入流程如图 2-6 所示：

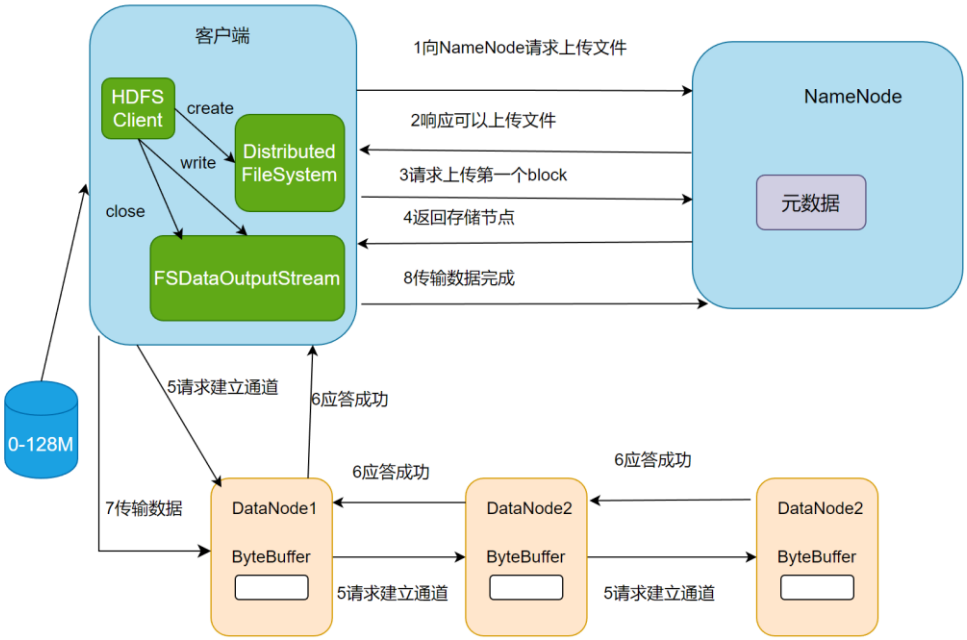

图 2-6　HDFS 写入流程

在 HDFS 写数据过程中，NameNode 会选择距离待上传数据最近的 DataNode

中国知网　https://www.cnki.net

接收数据。这个距离是两个节点到达最近的共同祖先的距离总和。

HDFS 读数据流程：客户端通过 DistributeFileSystem 向 NameNode 请求下载文件，NameNode 首先通过检索元数据信息以确定目标文件块所驻留的 DataNode 服务器地址。随后，从中选择一个 DataNode 服务器发起读取数据请求。接收到请求的 DataNode 随即开始从磁盘中读取相关数据流，并采用 Packet 作为校验单元进行数据传输。与此同时，客户端以同样的 Packet 为基本单位接收数据，首先将其暂存于本地缓冲区中，然后再逐步写入目标文件。这一流程如图 2-7 所示，清晰地描绘了从元数据查询直至数据写入目标文件的全链条过程。

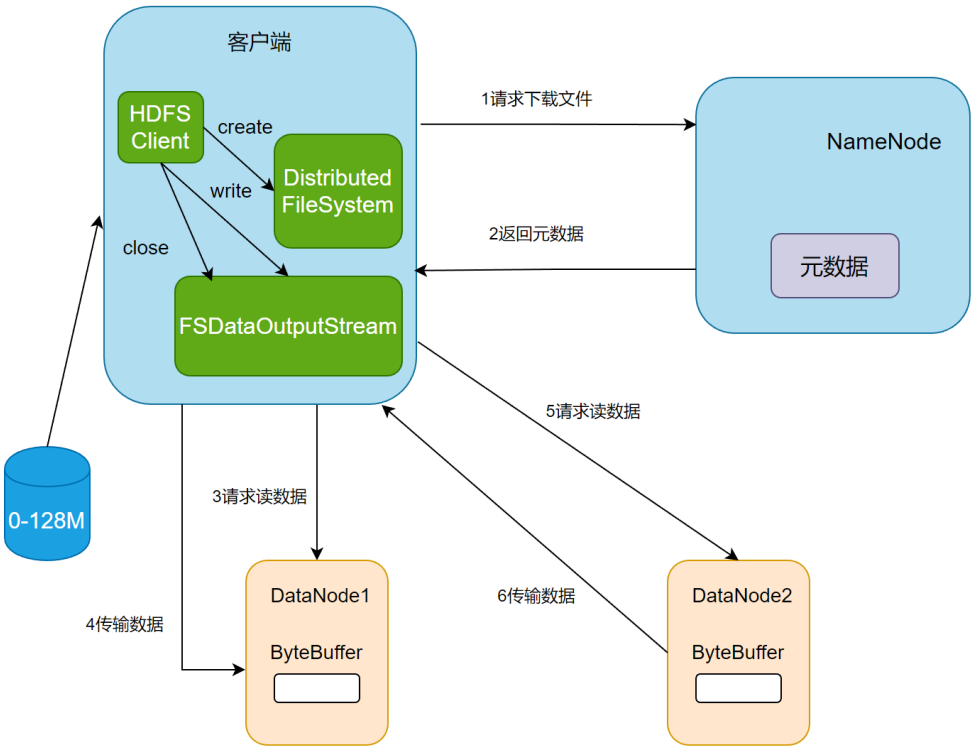

图 2-7 HDFS 读流程

NameNode 中的元数据存储：元数据是存在放在内存中。为了防止元数据丢失，还需要将元数据用 Fslmage 存储到磁盘中。为了保证元数据的一致性，HDFS 中存在一个 SecondaryNameNode 节点，工作流程分为两个阶段。第一阶段 NameNode 启动：1）第一次启动 NameNode 格式化后，创建 Fslmage 和 Edits 文件。如果不是第一次启动，直接加载编辑日志和镜像文件到内存。2）客户端对元数据进行增删改的请求。3）NameNode 记录操作日志，更新滚动日志。4）NameNode 在内存中对元数据进行增删改。第二阶段 SecondaryNameNode 工作：1）SecondaryNameNode 询问 NameNode 是否需要 CheckPoint。直接带回 NameNode 是否检查结果。2）SecondaryNameNode 请求执行 CheckPoint。3）NameNode 滚动

中国知网 https://www.cnki.net

正在写的 Edits 日志。4）将滚动前的编辑日志和镜像文件拷贝到 SecondaryNameNode。5）SecondaryNameNode 加载编辑日志和镜像文件到内存，并合并。6）生成新的镜像文件 fslmage.chkpoint。7）拷贝 fslmasge.chkpoint 到 NameNode。8）NameNode 将 fslmage.chkpoint 重新命名成 fslmage。如图 2-8 所示：

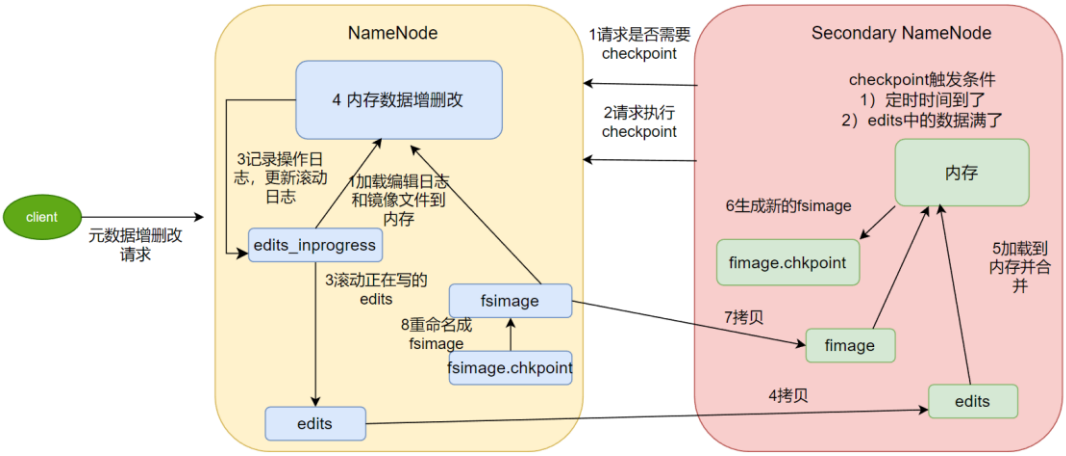

图 2-8 NodeName 节点元数据备份

DataNode 节点保证数据完整性的方法：1）当 DataNode 读取 Block 的时候，它会计算 CheckSum。2)如果计算后的 CheckSum,与 Block 创建时值不一样，说明 Block 已经损坏。3）Client 读取其他 DataNode 上的 Block。4）校验算法。常见的校验算法 crc(32) 如图 2-9 所示,md5（128），sha1(160)。5）DataNode 在其文件创建后周期验证 CheckSum。

图 2-9　crc 校验

中国知网　https://www.cnki.net

2.3 数据存取性能优化相关研究工作

大数据场景中的数据计算对写入性能方面至关重要，本文数据湖集成 Spark 计算引擎，针对 Spark 计算引擎性能原理方面进行了研究。大数据查询与实际应用息息相关，所以本文针对大数据领域查询性能方面进行了分析。

2.3.1Spark 性能优化研究

Spark 性能高的原因之一是弹性分布式数据集（RDD）。在分布式计算中，为了有效地处理数据，必须执行一系列核心功能，包括数据分区、跨分区的数据传输、数据存储以及数据计算等。为了实现这些功能的统一管理和协调，需要引入一个核心的数据抽象概念。这个概念就是 RDD。通过 RDD，Spark 能够高效地处理大规模数据集，并充分利用集群资源进行计算。从 RDD 的源码中可知，RDD 具有五大属性。1、a list of partitions，Spark 中的 RDD 是被划分为多个分区的，这些分区分别由不同的计算任务进行处理，而分区的数量直接决定了并行计算任务的并发度，而且 RDD 之间的并行处理层级是可以延续的。在默认配置下，HDFS 中一个数据分片在 Spark 中被视为一个分区，而 RDD 分区的数量直接影响着并行计算的粒度和强度。在初始化 RDD 时，用户可以选择自定义 RDD 分区的数量以适应特定的并行处理需求。如果没有特意设定分区数，RDD 的分区数取决于其创建方式。若 RDD 是从内存集合中构建而成，其默认分区数将与当前程序所分配到的 CPU 核心数相符，通常每个 CPU 核心可以对应承载 2 至 4 个分区。相反，若 RDD 是由 HDFS 文件创建，其默认分区数则与该文件在 HDFS 中的数据块（Block）数量保持一致。2、a function for computing each split，每个分区都会有计算函数，Spark 的 RDD 的计算函数是以分片为基本单位的，每个 RDD 都会实现 compute 函数，对具体的分片进行计算，不需要保存每次计算的结果。RDD 中的分片是并行的，所以是分布式并行计算。有一点非常重要，就是由于 RDD 有前后依赖关系，遇到宽依赖关系，如 reduceByKey 等这些操作是划分成 Stage，Stage 内部的操作都是通过 Pipeline 进行的，在具体处理数据时它会通过 Blockmanager 来获取相关的数据，因为具体的 split 要从外界读数据，也要把具体的计算结果写入外界，所以用了一个管理器，具体的 split 都会映射成 BlockManager 的 Block，而具体的 split 会被函数处理，函数处理的具体形式时以任务的形式进行的。3、a list of dependencies on other RDDS，由于 RDD 的每一次转化操作都会生成全新的 RDD 实例，因此 RDD 之间会构建起一种链式依赖结构，尽管这一比喻在宽依赖场景下并不完全等同于流水线作业模式。对于宽依赖类型的连接，后续 RDD 的具体分区数据完整性依赖于其上游所有 RDD 的所有分区数据，这就打破了数据分片在内存层面串行流水线处理的可能性，转而倾向于

中国知网　https://www.cnki.net

跨多台机器分布式的处理方式。鉴于这种明确的前后依赖联系，一旦某个分区数据发生丢失，Spark 框架能够依据依赖图谱追溯并重新计算所需的数据，而非对整个 RDD 的所有分区进行全面重算。RDD 间的依赖关系主要分为两类：一是窄依赖（Narrow Dependency），二是宽依赖（Wide Dependency）。如图 2-10 所示：窄依赖是指每个父 RDD 的一个 Partition 最多被子 RDD 的一个 Partition 所使用，这种不会引起 shuffle 操作，速度快。宽依赖是指一个父 RDD 的 partition 会被多个子 RDD 的 partition 所使用，这种操作会引起 shuffle 操作，速度慢。针对数据丢失情况，系统能够利用 RDD 间的内在依赖性逻辑，仅针对缺失数据的相应分区执行重新计算操作，而无需对整个 RDD 的所有分区进行全局回溯式的重新计算处理。这种有针对性的故障恢复机制得益于 Spark 对 RDD 之间依赖关系的深刻理解和有效利用，从而实现了在保证数据完整性和一致性的同时，最大程度地减少了因数据丢失所带来的额外计算负担。4、a Partitioner for key-value RDDS,即 RDD 的分区函数。5、a list of preferred locations to compute each split on,存储每个切片优先位置的列表。基于大数据处理领域广泛认同的“数据搬运成本高昂，优先移动计算”的原则，Spark 在任务调度过程中，会参考精心策划的优先位置列表，尽量将计算任务部署在所需处理数据的实际位置。这种策略确保了计算任务与数据资源的就近匹配，从而实现了高效协同。通过优化任务与数据的布局，Spark 显著减少了不必要的数据搬运，提高了整体处理效率，为大数据应用提供了更加稳定和高效的运行环境。

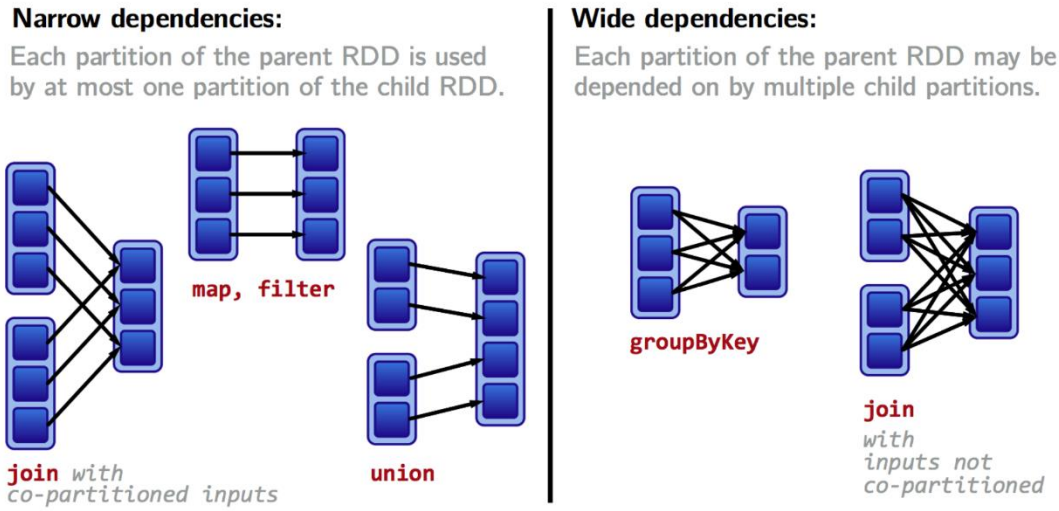

图 2-10 RDD 之间的依赖关系

2.3.2 大数据查询性能优化研究

数据湖本质上可以理解成一种海量数据库。针对数据湖性能优化研究方向可以从数据库查询性能优化方向研究出发。研究表明，从用户提交一条 SQL 语句进

中国知网　https://www.cnki.net

行查询到最终输出结果的过程中，大量的时间是消耗在磁盘 I/O 上，在很多情况下，I/O 所占用的时间可以达到整个时间的 90%以上。因此，减少磁盘 I/O 对查询速度的影响是进行查询性能优化的一个关键方向。

有研究提出数据预排序处理，在数据写入磁盘前进行排序，以保证数据在磁盘上有序，即数据在数据文件中是有序的。这种处理方式在数据库系统中已经被广泛使用，在实现范围查找时，可以将大量的随机读转换为顺序读，在顺序读过程中可以采用多种优化算法（如二分查找算法）进行查询优化，从而有效提高 I/O 效率，降低范围查询的 I/O 时间。在点查找时，预排序能够做到和未排序数据相同的性能。因此，预排序可以在不降低点查询性能的情况下有效提高范围查询的性能。同样是为了减少磁盘 I/O 时间，有研究提出数据压缩方案。数据压缩可以减少读取和写入的数据量，从而减少 I/O 时间。但压缩必然带来压缩和加压缩的 CPU 消耗，这是一个里利用 CPU 时间换 I/O 时间的手段。典型的应用是 ClickHouse 组件，在 ClickHouse 中数据处理最小的单元是块，块一般有 8192 行数据组成，ClickHouse 的一次压缩针对的是同一列的 8192 行数据，ClickHouse 采用的是列式存储，同一列的数据相对更有规律，因此能够带来比较大的压缩比。列式存储方式在 OLAP(On-line Analytical Processing,联机分析处理)场景对性能有较大的提升。列式存储数据库和行式存储数据库最根本的区别在于列式数据库将一行数据拆分到多个数据文件中。在列式数据库中，同一列的所有数据都在同一个文件中，因此在硬盘上是连续的。这种特性特别适合 OLAP 的低范式查询场景。目前常见的采用列式存储方式的大数据组件有：HBase、ClickHouse、Druid 等。

2.4 本章小结

本章首先对数据湖进行概述，包括数据湖基本概念、数据湖发展现状、数据湖典型技术。其次介绍了本文中数据湖存取涉及到的关键技术。最后，从 Spark 计算引擎优化原理和大数据查询性能优化这两方面介绍了数据存取性能优化相关研究工作。

中国知网　https://www.cnki.net

第 3 章 数据湖访问模式优化

3.1 引言

数据湖的访问模式是指用户对数据湖中数据的访问方式。根据数据湖中数据的存储方式和应用场景的不同，可以采用不同的访问模式。主要的两种数据湖访问模式：1)批处理访问模式：批处理访问模式是指对数据湖中数据进行批量处理和分析的方式。用户可以通过提交批量作业的方式，对数据湖中的数据进行处理和分析。这种访问模式适用于对数据的实时性要求不高，但对数据分析的准确性和深度要求较高的场景。2)流处理访问模式：流处理访问模式是指对数据湖中数据进行流式处理和分析的方式。可以通过实时流处理技术，对数据湖中的数据进行实时处理和分析。这种访问模式适用于对数据的实时性要求较高的场景[40-41]。

在实际应用中，不同业务场景和不同硬件设备情况下，两种访问模式均会产生数据分布不均和缓存利用率低的问题。针对数据湖的两种访问模式下产生的问题，本文研究分布式数据处理机制的优化技术。

在构建和管理数据湖时，确保数据的均匀分布是优化系统性能、稳定性和可扩展性的重要手段。数据湖通常采用分布式存储架构（如 Hadoop HDFS 或 Apache Hudi 等），在这种架构中，数据被分割成多个部分并分散存储在集群中的各个节点上。哈希算法在这里起到了关键作用。通过将数据的关键字段（如用户 ID、事件时间戳等）进行哈希运算，可以将数据映射到一个确定的范围或者特定的桶（bucket）中，进而决定其在集群中的存储位置。由于哈希函数理论上能够实现数据的均匀分布，因此这种方法有助于确保每个节点存储的数据量大致相等，避免了“热点”问题，即某些节点因存储过多数据而造成资源紧张，其他节点却闲置的情况。例如，在 Hadoop 的 HDFS 中，通过 Block 的哈希值分配策略，可以使得相同 Key 的数据倾向于落在同一 DataNode 上，从而提高数据读写的局部性；在数据库分片（Sharding）场景下，也可以利用哈希算法对表进行水平切分，使得各分片数据量均衡。此外，对于实时计算和流处理系统，如 Apache Kafka 或 Flink，也常常利用哈希算法对消息或事件进行分区，以保证数据在多个消费者实例之间均匀分配处理，防止单点过载，提高系统的整体处理能力和稳定性。

在数据湖查询过程中，用分布式数据处理缓存机制可以减少磁盘 I/O,并提高数据湖的性能。文献[42]中提出了 DryadInc，基本架构由“缓存服务器”(Cache Server)和“重执行逻辑”(Rerun Logic)组成[42]。在 Spark 计算环境中，数据处

中国知网 https://www.cnki.net

理的核心环节被设计为以内存为中心，借此大幅度削减了对磁盘 I/O 的依赖性。这样的设计策略不仅消除了频繁的数据序列化开销，还显著减少了大量的 I/O 操作所带来的性能损耗。同时，Spark 在内存空间中缓存部分中间结果，下次使用这些中间结果时可直接在缓存中访问[43]。本文基于 Spark 内存计算原理，在数据湖分布式场景下，选择了将 Spark 的缓存机制与数据湖相结合，并利用 LRU 算法实现数据湖的缓存机制。另一方面，数据预处理和数据湖索引对数据湖查询性能有质的飞跃，通过精心设计的数据预处理流程以及高效合理的索引策略，可以在保障数据质量的同时，显著提升数据湖的查询性能，使其真正成为企业数据分析与挖掘的强大工具。

综上，本文提出了基于数据分区和缓存机制的访问优化方案，以提高数据湖的性能。

3.2 基于数据分区和缓存机制的访问优化方案

3.2.1 基于哈希算法的数据均匀分布

为确保在处理大规模数据时维持高效能，有必要对数据存储进行分区布局。在存储系统中，分区作为一种应对极端大型表格的有效策略被广泛应用。该技术旨在规避将整张表格视作单一且庞大的实体进行统一管理的做法，同时为了提高海量数据的可扩展性性能，本文采用了一种策略，即将大型表和索引细分为易于管理的小型单元。这种分区策略不仅显著减少了管理操作所需的时间，而且还增强了系统的可用性。通过实施分区，系统能够局部化处理故障数据，从而有效屏蔽故障部分。此外，本文还强化了并行处理能力，以进一步提升整体性能水平。这种综合策略使得系统在面对大规模数据集时，能够保持高效、稳定和可扩展的性能表现。通过将操作分配到更小的存储单元。在传统数据库管理系统中，分区表的分割策略种类繁多，其中范围分区算法作为一种历史悠久且广为人知的经典方法，主要是依据分区字段值的区间范围来实现数据分割。这种方法尤其适用于那些按照时间序列进行存储的数据场景，通过范围分区，能够极大地增强对数据迁移、备份及其他相关管理操作的便利性和效率。然而，应当注意的是，分区策略与记录值之间存在着内在联系，这可能导致数据在不同分区间的分布不均匀。由此，分区可能会带来执行效率的挑战和维护难度的上升，特别是在保持数据分布均衡及系统高效运维方面可能存在一定劣势。通过分区的离散值进行分区，适用于数据的离散值控制，只支持单个域的数据离散值控制，虽然易于执行，总体表现最佳，但对数据的管理能力较弱，不能控制列表分区，因此，这种分区方法适用于静态数据的均匀存储，有优缺点与范围分区相似，但分区的数据未必均匀，

中国知网 https://www.cnki.net

且与记录值相关，执行难度与可维护性相对较差。有两种类型的组合分区：Range-Hash 和 Range-List，它们既适用于历史数据，也适用于数据的均匀分布，能够像范围分区一样提供高可用性和管理性[39-40]，具备处理海量数据的可能性的正是这些对数据库表分区的技术。范围分区是一种基于预设及用户可自定义数据值域范围来划分表结构的分区策略。其核心理念在于依据特定的数据值域边界设定，将表数据划分为不同的分区段落。简言之，范围分区的界定依据即是用户根据实际需求所确定的数据范围界限，用于指导表数据的细分与组织。其边界是由开发人员选择的，也可以随着数据使用方式的变化而变化[44-45]。根据特定的和可定制的数据范围定义的表分区，通常是按照日期来划分。 旧资料可通过范围分区快速存档、替换[46-47]。在这种情况下，数据所处的特定位置是不可缺少的，所以相应的分区只有在需要时才能访问到。一般情况下，在决策支持大范围数据时，数据访问通常使用范围分区最适合[48]。

在传统的数据湖分区方法中，普遍采纳了负载均衡策略以实现数据划分，然而，此方法忽视了一个关键缺陷，即未充分考虑集群环境中各节点的具体状况，诸如单个节点的 CPU 使用率、内存容量及磁盘空间等实际参数。鉴于集群中各节点的硬件配置可能因实际条件而异，单纯依赖统一的负载均衡规则进行分区，可能会导致部分节点过载而其他节点资源闲置的现象，从而未能有效利用整个集群的资源潜力。为应对这一挑战，有必要实施一种机制，能够动态监测集群的各项性能指标，并据此动态调整数据的分区策略，以达到资源优化配置的目的。本文提出了一种改进的哈希算法，该算法通过为集群中的每个节点设定 CPU、内存和磁盘的优先级顺序，其中 CPU 优先级最高，内存次之，磁盘最低，并依据具体指标进行数据分配决策。具体而言，算法偏好将数据首先分配至 CPU 空闲率最高的节点，其次考虑内存利用率最低的节点，最后选择磁盘空闲率最高的节点。这意味着算法执行逻辑遵循严格的优选顺序：首要考量 CPU 空闲资源，若所有节点的 CPU 条件均不满足分配标准，则降级考虑内存利用率最低的节点；若 CPU 与内存条件均不适宜，最终选取磁盘空闲率最高的节点作为数据分区的目标。通过这一系列精细化的评估与选择过程，旨在实现更高效、更均衡的集群资源利用。

哈希算法如图算法 1 所示，包括以下步骤：1）选择分区键：首先确定一个或多个字段作为数据分区的依据，这些字段的选择应当尽量使得数据在整个键空间内均匀分布。2）计算哈希值：对每个记录的分布键应用哈希函数，将其转化为一个固定长度的哈希值。3）重新生成分区键：根据哈希值，并结合实际节点数量，对哈希值进行取模运算，得到新的分区键。4）分区分配：根据新生成的分区键，并且根据集群节点的优先级将数据分配到不同的集群节点上。通过上述步骤，即使随着节点数量的变化，也可以通过调整哈希映射规则来继续保持数据

中国知网 https://www.cnki.net

在新增或减少节点上的相对均衡分布。这种方法对于大数据环境下的数据湖应用具有很高的实用价值，有助于提高查询性能和系统的整体稳定性。

算法 1 数据均匀分布算法

```
输入：数据集 df，分区键 pk，集群节点数 n;
输出：新分区键 npk;
1 df; //原始数据集；
2 pk; //分区键；
3 n; //集群节点数 n；
4 for each df in i
   npkey = (md5(df[i].pk))mod(n); //对分区键取 md5 函数取模运算生成新的哈希值；
   df[i].pk = npkey;//生成新的分区键
  end for；
5 for each df in i
      for each n in j
         if(CPU) //CPU 优先级
            df[i].key == j
            insert(); //遍历数据集，根据新的分区键写入集群对应的节点
         else if(memery) //内存优先级
               df[i].key == j
               insert(); //遍历数据集，根据新的分区键写入集群对应的节点
            else   //磁盘优先级
                  df[i].key == j
                  insert(); //遍历数据集，根据新的分区键写入集群对应的节点
         end if；
      end if；
   end for;
  end for；
```

3.2.2 缓存机制

缓存技术作为一种广泛应用于提升系统性能的有效手段。同样的缓存策略，鉴于缓存存储资源的有限性以及各异的数据访问模式，学术研究中涌现了多种旨在满足多元业务需求的缓存策略方案。然而，由于各类系统数据访问行为的差异性，这些策略难以在所有场景下均能达到理想的性能表现。置换时以来访时间为准。 根据每个缓存项最近的访问情况来决定替换对象的这样一种算法组织缓存结构，也称为时间上的邻近性，比如经典的 LRU 策略 [42]，但是在应用程式的工作集不适合缓存的情况下， LRU 会出现频繁的抖动，从而导致性能突降。 为了克服 LRU 的性能弊端，学术界研究了许多不同应用场景的变种，例 如 ： 多 倍 最 近 最 少 使 用 (AM-LRU ， AM-LRU)[43] ， 最 近 最 少 使 用(LRU BASED ON USER BEHAVIOUR，LRU-UB)[49]，基于用户行为的变种。最近使用最少的基于计

中国知网 https://www.cnki.net

数滤镜的算法(CCF-LRU，CCF-LRU)[50] 等，主要改进方向为 3 个技术。文献[51]改进了基于 Belady 演算法的缓存替换，通过使用过去的缓存历史访问来解释如何学习 Belady 演算法来预测未来的缓存替换决策，提出的新的缓存替换策略由两部分组成，一部分是最好的解决方案重建 Belady 访问过去的缓存(cache)；二是实现一种可以学习计算机系统决定将过去的行为驱逐出未来的负载模式的预测器(Preductor)[51]。著名的 LRU-K 和 2Q(2Q，Two queues algorithm)是基于访问频率思维的模式。

Spark 是基于内存计算的，这是 Spark 高转速的最关键理由。通常情况下，它所需要的 RDD 和它的前几代 RDD 在遇到动作操作时都会被计算出来，但如果同样的 RDD 需要用到几个动作时，为了避免多次重复计算，可以考虑暂时保存这个 RDD。在 Spark 中，RDD 可以使用不同的存储等级(Storage Level)缓存，既可以将 RDD 保存在磁盘上，也可以将 RDD 保存在内存中，还可以将 RDD 存储为 JAVA 对象的序列化(Space)，从而实现对其的跨节点复制。RDD 的默认缓存界别是 Storage Level. NONE，实现 RDD 缓存需要修缓存级别，在 Spark 中有多种缓存级别[46]，如表 1 展示了 Spark RDD 的不同持久化级别。

表 1 Spark RDD 持久化级别

级别	内存使用率	CPU 耗时	是否存储在内存	是否存储在磁盘
MEMORY_ONLY	high	low	yes	no
MEMORY_ONLY_SER	low	high	yes	no
MEMORY_AND_DISK	high	middle	some	some
MEMORY_AND_DISK_SER	low	high	some	some
DISK_ONLY	low	high	no	yes

在面对内存容量有限的情况时，Spark 采用了 LRU（最近使用最少）算法来优化内存管理。该算法旨在清除内存中未使用时间最长的 RDD，从而确保内存的有效利用。Spark 提供了五种不同的持久度等级，这些等级决定了 LRU 算法在处理被淘汰 RDD 时的行为。对于前两个等级，被淘汰的 RDD 会被直接清除出内存，这意味着如果未来需要重新使用这些数据，系统将不得不重新计算它们。而对于 LRU 算法淘汰的第三和第四种持久度等级的 RDD，它们会被写入磁盘进行持久化存储。虽然这些数据在磁盘上仍然可用，但重新使用时需要从磁盘中读取，这增加了额外的磁盘 I/O 操作，从而对 Spark 的执行速度造成一定影响。值得注意的是，第五种持久度等级则是一种特殊的策略，它将需要缓存的 RDD 数据直接存储在磁盘中，而不是占用有限的内存资源。这种策略可以在内存不足时提

中国知网 https://www.cnki.net

供额外的存储空间，但同样可能增加磁盘 I/O 操作。总体而言，不同的持久度等级对 Spark 执行速度的影响是不同的。在选择合适的持久度等级时，需要综合考虑内存可用性、磁盘 I/O 性能以及数据的重用频率等因素，以优化 Spark 应用程序的整体性能。

在数据湖应用中，本文采用内存缓存策略，使用 Spark 的 RDD 缓存机制将数据在存储在内存中[49-50]。RDD 在 Spark 中的典型执行过程包括以下步骤：1、通过读取外部数据源创建 RDD，利用 TestFile 功能加载本地数据；二、RDD 每做一系列转换变形操作后，下一次转换时会有不同的 RDD 生成。其中，变形操作可以使用费勒函数；3、经过“动作”动作操作，最后一个 RDD 转换输出到外接数据源。如图 3-1 所示：

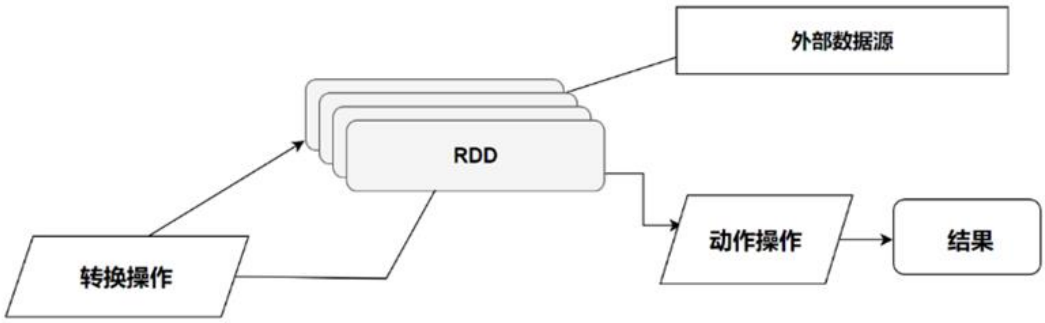

图 3-1　RDD 运行过程图

在架构数据湖的缓存模型过程中，采用了一种系统化的方法来集成 Spark 的弹性分布式数据集(Resilient Distributed Dataset, RDD)与 Hudi 数据湖技术，具体通过以下四个有序阶段来实现这一集成：1）数据摄取初始化，通过 Hudi 的数据访问接口从已有的 Hudi 数据集中提取数据。这一阶段涉及读取存储于 HDFS 或其他兼容 Hadoop 文件系统的 Hudi 数据，为后续的处理流程奠定基础；2）RDD 实体构建，利用 Spark 的高性能计算能力，创建出代表大规模数据集的 RDD，这一转换步骤确保了数据的分布式特性，并为灵活的数据操作提供了基础；3）RDD 至 Hudi 的映射与持久化，核心步骤在于运用 Hudi 专为 Spark 设计的数据源 API，将构建好的 RDD 转换并写回到 HDFS，期间转换为 Hudi 数据格式。这一转换不仅保留了 RDD 的原始数据结构，还利用 Hudi 的特性实现了数据的增量更新与删除功能，增强了数据管理的效率与灵活性；4）高级数据分析与处理：最后，借助 Spark 的丰富 API 集合，对转换为 Hudi 数据集的信息执行高级分析与处理任务。这包括但不限于聚合、过滤、转换等复杂操作，旨在深度挖掘数据价值，支持即时查询与大数据分析需求。通过这些步骤，基于 Spark 的 RDD 缓存机制实现数据

中国知网　https://www.cnki.net

湖的缓存。将 Spark 的 RDD 与 Hudi 相结合的伪代码如算法 2：

算法 2 Spark 的 RDD 与 Hudi 结合算法

输入：Hudi 数据集 hudiDF;
输出：转换后的 Hudi 数据集 hudiDFt;

```
1 hudiDF; //Hudi 原始数据集
2 hudiDFt; //转换后的 Hudi 数据集
3 rdd; //Spark 的 RDD
4 hudiRdd;//Hudi 中的 RDD
5 filteredDF;//对转换后 Hudi 数据集进行计算处理
6 hudiDF = Spark.read.format(); //读取 Hudi 数据集
7 rdd = sc.parallelize(hudiDF); //创建 Spark 的 RDD
8 Val hudiOptions = Map[String,String](HoodieWtriteConfig.TABLE_NAME -> “hudiTest”); //创建 Hudi 表
9 hudiRdd=spark.sparkContext.parallelize().map{case(hudiId,hudiPation,value) => (id.toString,(hudiId,hudiPatition,value))}; //将 Spark 的 RDD 转换为 Hudi 数据集
10 hudiRdd.toHudi(); //写入 HDFS
11 hudiDFt = Spark.read.format();//从 HDFS 中获取转换后的 Hudi 数据集
12 filteredDF = hudiDF.filter();//使用 Spark 对 Hudi 数据集进行处理
```

3.2.3 索引优化与预处理技术

在数据湖应用中，实时或近实时地进行大量的求和、平均值、最大值、最小值等数据聚合分析会面临性能挑战，尤其是当数据量巨大且查询复杂性较高时。这是因为数据湖通常以原始形式存储大量数据，查询和处理这样的数据可能需要消耗大量计算资源和时间。为了改善查询性能，一种有效的策略是预先对数据进行聚合操作，也就是在数据进入数据湖后，或者在查询前对其进行预处理，对常见的聚合指标提前进行计算并存储，如每日、每周或每月的总计、平均值等，并将结果存放在优化过的表结构中。可以在很大程度上减少查询过程中的计算量，显著提升数据湖在应对聚合分析场景时的性能表现。

在进行数据湖模型构建过程中，索引优化是一个重要环节，直接影响数据的查询和处理效率。本文采用分布式文件系统 HDFS 作为数据湖的存储结构。数据写入 HDFS 中是以小文件的形式存在[59]，也可称之为数据分区。在数据分区不创建索引的情况下，数据查询时将需要对所有数据分区进行遍历操作，会降低查询效率。为了减少对分区的遍历操作，本文采用块级索引的策略，对数据分区创建索引，可以快速定位包含关键字的数据分区，提高查询效率和响应速度。

中国知网　https://www.cnki.net

3.3 实验与分析

3.3.1 实验环境

Spark 运行平台：Spark 计算层设置 7 个节点，其中 1 个主节点，6 个工作节点，每个工作节点 32GB 内存，16 个 cpu 内核（Intel i5-8500 处理器）。HDFS 分布式文件系统：HDFS 存储层设 8 个节点，其中 1 个主节点，7 个数据节点，8 个节点均为 32GB 内存，16 个 cpu 内核（Intel i5-8500 处理器）。Hudi 数据湖：4 个节点，4 个节点均为 64GB 内存，32 个 cpu 内核（Intel i5-8500 处理器）。

3.3.2 数据集

实验数据共有 11 个样本文件，其存储格式均为 Apache Parquet。每个样本会带有版本号，样本文件名会体现版本信息，如 base-0.parquet、base-1.parquet，分别代表当天产生的离线数据，版本号越大表示生成数据越新，如 base-1 比 base-0 的数据晚一天；每一个版本的数据字段数量一致，不同版本数据 uuid 会有部分相同。数据格式如表 2。其中，文件 base-0.parquet 是数据量最大的文件，数据量大小分布如表格 3。

表 2 数据字段以及类型

字段值	uuid	ip	hostname	request	name	city	job	phonenum
类型	string	string	string	int	string	string	string	string

表 3 数据量大小分布

文件名	数据量大小（MB）
base-0.parquet	1982
base-1.parquet	395
base-2.parquet	794
base-3.parquet	394
base-4.parquet	583
base-5.parquet	293
base-6.parquet	476
base-7.parquet	732
base-8.parquet	467
base-9.parquet	483
base-10.parquet	584

3.3.3 实验模型构建

利用 Spark、HDFS 和 Hudi 进行模型构建，具体流程如图 3-2 所示，主要包括以下四个步骤：1)获取数据集文件；2)进行数据分区；3)基于 Spark 集群，进行并行调度将数据文件写入 HDFS；4)创建索引:对写入 HDFS 中的分区构建分区

中国知网 https://www.cnki.net

索引。

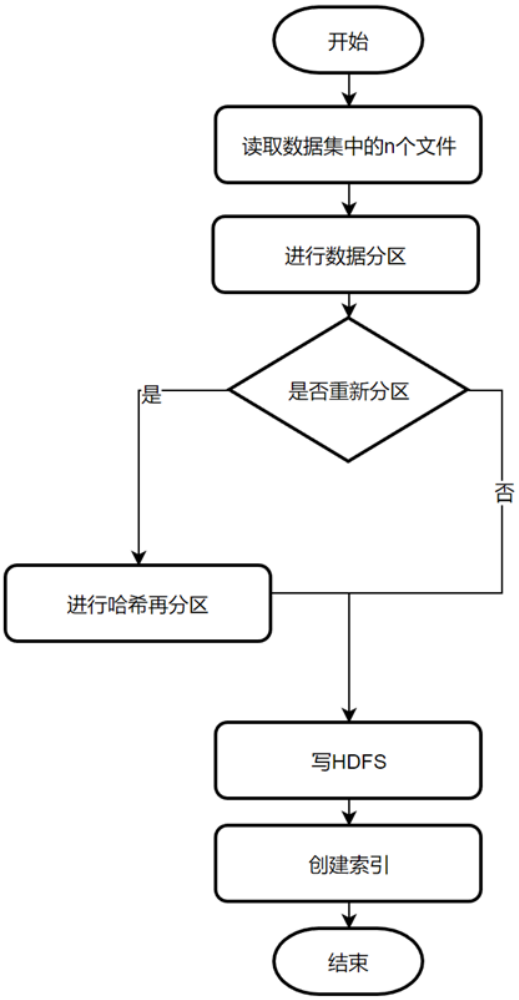

图 3-2　实验模型构建流程图

3.3.4 基于哈希算法的数据均匀分布实验

本文设计并实施了两项实验，均依托于上述数据集展开。实验 1 旨在建立基准线表现，通过利用数据集中的 uuid 字段实施数据分区，但在此过程中并未考虑集群的具体硬件配置（包括 CPU 能力、内存大小及磁盘空间），直接执行数据的写入与查询操作。实验 2 构建于实验 1 的基础上，同样以 uuid 字段为分区依据，显著区别在于融入了本文所提出的哈希分区算法。该算法依据节点的 CPU 空闲率、内存利用率及磁盘空闲率，实施了更为精细化的数据分配策略。通过这一机制，实验 2 在进行数据写入与查询操作的同时，实现了对集群资源的动态适应与高效利用。关于实验环境的具体配置，特别是 Hudi 数据湖集群的状态，图 3-3 直观反映了集群中 CPU 空闲率，图 3-4 直观反映了内存利用率。

中国知网　https://www.cnki.net

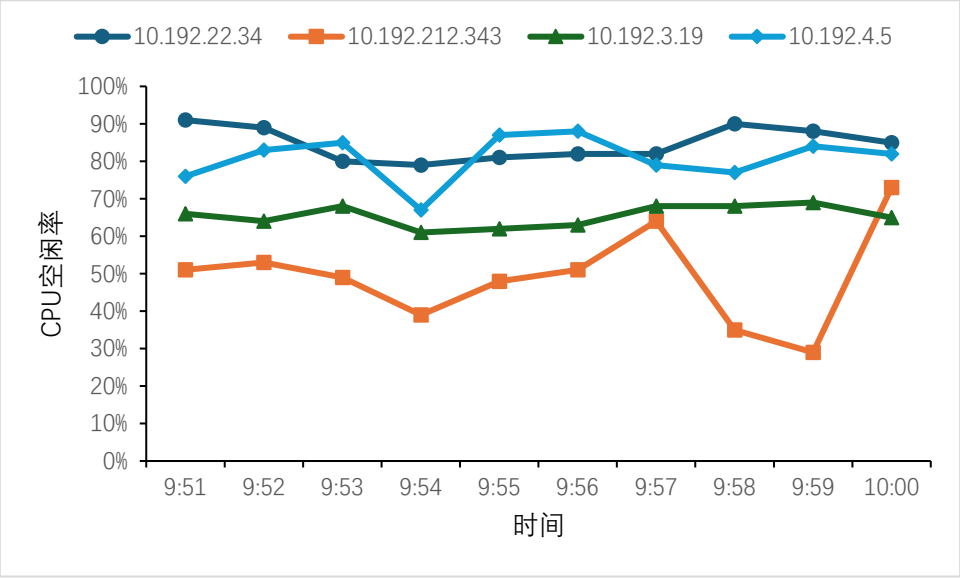

图 3-3 CPU 空闲率

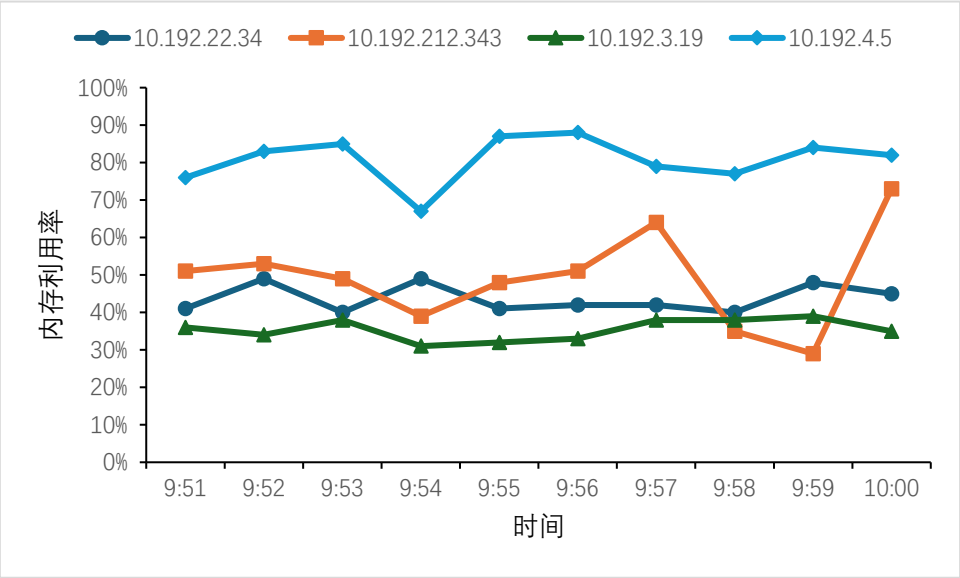

图 3-4 内存利用率

为了增强实验结果的稳定性和可靠性，针对每组实验进行了三次独立运行并计算了相应操作的平均耗时，实验结果如表 4。实验 1 在数据写入环节的平均执行时间为 733.6 秒，数据查询的平均耗时则为 282.6 秒。实验 2 应用了本文提出的哈希分区算法后，数据写入的平均时间缩短至 577.3 秒，查询操作的平均时间也减少到了 243.6 秒。对比两组实验的平均结果，可以明确观察到，采用本文的哈希分区算法在数据处理效率上呈现出显著提升。数据写入操作的平均时间减少了约 21.6%，而数据查询的平均时间也缩减了约 13.8%。这一系列量化数据有力地证明了，哈希分区算法在面对实际数据处理任务时，能够有效优化资源分配，加速数据的写入与检索过程，进而整体提高了数据处理的效能。实验结果表明，

中国知网　https://www.cnki.net

结合集群硬件状况的动态数据分区策略对于提升数据湖环境下数据处理效率具有重要价值。

表 4 基于哈希算法的数据均匀分布实验结果

实验名称	写入时间/s	平均值/s	查询时间/s	平均值/s
实验 1	834	833.6	283	282.6
	833		282	
	834		283	
实验 2	578	577.3	243	243.6
	577		244	
	577		244	

3.3.5 缓存机制实验

为了系统性地评估缓存机制对查询性能的影响，设计并执行了两项对照实验，均基于相同的初始数据集并针对一个特定查询操作。具体而言，实验 1 作为一种基线测试，未采取任何缓存策略，直接对数据集执行查询操作。此设置意在确立无缓存介入情况下的查询性能基准。实验 2 则融入了本文提出的缓存机制，旨在通过利用预存储的数据副本加速查询过程，减少直接从主存储中检索数据的需求。这一实验设计的核心目标是验证所提案缓存策略的有效性及其对查询效率的潜在提升。为确保实验结果的准确性和可重复性，每项实验均进行了三次独立运行，以此消除偶然误差，获取更加稳健的平均性能指标。如表 5 所示，在未采用缓存机制的实验 1 中，查询操作的平均执行时间被记录为 452.3 秒；应用了本文提出出的缓存策略的实验 2 中，这一平均耗时显著降低至 239 秒。通过对比这两组实验的结果，本文提出的缓存机制对查询性能有显著的提高，在数据密集型应用中具有实用价值。

表 5 缓存机制实验结果

实验名称	查询时间/s	平均值/s
实验 1	456	452.3
	444	
	457	
实验 2	238	239
	240	
	239	

3.3.6 索引优化与预处理实验

针对特定的聚合查询场景，利用选定的数据集进行两项实验设计，聚焦于提升查询效率与性能。实验以 uuid 作为维度字段，对 request 字段执行聚合操作，具体如下：实验 1 作为一种基本场景，直接在原始数据集上执行聚合查询，未采取额外的优化措施。此设置用以建立未经特别处理条件下的查询性能基准。实验

中国知网　https://www.cnki.net

2 采用本文提出的策略，事先对聚合数据进行了预处理，并针对查询涉及的关键字段创建了索引。这一预处理步骤旨在通过减少查询时的数据扫描范围与提升数据访问速度，来显著增强查询效率。如表 6 所示，实验数据分析清晰地揭示了两种查询策略之间的性能差异：实验 1 中，聚合查询的平均执行时间记录为 548 秒；而在采取了数据预处理及索引创建措施的实验 2 中，这一时间大幅缩短至 139 秒。这种显著的时间缩减，直观地展现了预处理与索引机制在加速聚合查询方面的正面效应。通过深入对比这两组实验的结果，不仅验证了预处理步骤与索引构建对于提高聚合查询效率的积极影响，而且为探索和优化大规模数据环境下的数据管理与查询策略提供了坚实的实证基础。

表 6 缓存机制实验结果

实验名称	查询时间/s	平均值/s
实验 1	543	548
	544	
	557	
实验 2	138	139
	140	
	139	

3.3.7 数据湖访问优化方案实验

为了全面评估数据湖访问优化策略的性能表现，本文基于选定的数据集设计并实施了四项对比实验，旨在验证不同优化方案的有效性。具体实验如下，实验 1 作为基线测试，直接将数据集导入 Hudi 表中，并执行全表查询操作，未应用任何特定优化措施。实验 2 至实验 4 均融入了数据分区和缓存机制的优化方案。其中：实验 2 关注于数据集中异常大的首个文件 base-0.parquet，将其依据 uuid 字段进行数据分区，之后写入 Hudi 表并创建相应的分区索引。数据集经由 Hudi 数据源 API 转换为 Spark RDD，随后执行全表查询。实验 3 选取数据集中三个较大的文件（base-0.parquet、base-2.parquet 及 base-7.parquet），基于 uuid 字段实施数据分区，同样完成写入 Hudi、建立索引及 RDD 转换步骤，以进行全表查询，评估多大文件分区策略的性能提升。实验 4 则最为全面，对所有数据文件应用 uuid 字段的数据分区策略，完成写入 Hudi 表、创建分区索引以及 RDD 转换流程，最后执行全表查询，以探索全范围数据分区的最优化效果。为了确保结果的稳定性和可靠性，每一实验均重复进行三次，所有实验的写入时间和查询时间被详细记录。基于这些数据，进一步计算出每次实验的平均写入时间和查询时间，并汇总于表格 7 中，以便于直观对比不同优化方案下的性能差异。通过这一系列严谨的实验设计与分析，本文旨在深入探讨数据分区与缓存机制在提升数据湖访问效率方面的具体贡献，为优化大规模数据处理提供科学依据与实践指导。

中国知网　https://www.cnki.net

表 7 基于数据分区和缓存机制的访问优化方案实验结果

实验名称	写入时间/s	平均值/s	查询时间/s	平均值/s
	734		183	
实验 1	733	733.6	182	182.6
	734		183	
	478		143	
实验 2	477	477.3	144	143.6
	477		144	
	365		118	
实验 3	366	335.3	117	117.6
	365		118	
	333		102	
实验 4	334	333.6	103	102.6
	334		103	

图 3-5 所示的实验结果表明，采用数据分区与缓存机制的优化策略数据写入操作的最优耗时仅为 333.6 秒，而查询操作的最优时间缩短至 102.6 秒，相较于未优化的情况，本文提出的优化方案在数据湖存取效率方面实现了较为显著的增强。通过系统性对比各实验数据，数据分区与缓存机制的优化方案对提升数据湖的存储与检索性能具有决定性作用，有力地验证了该方案在提升大规模数据管理效率方面的有效性和实用性。

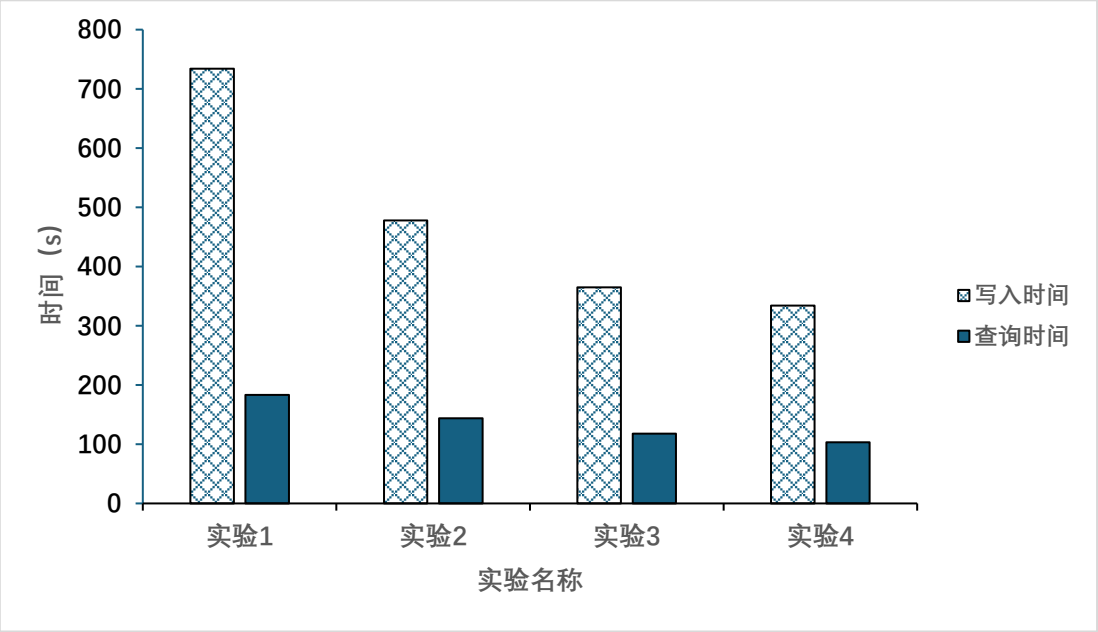

图 3-5 四种实验写入和查询时间

中国知网 https://www.cnki.net

3.4 本章小结

本章提出一种基于数据分区和缓存机制的访问优化方案，旨在提高数据湖性能。利用数据分区技术将数据分割成更小的子集，从而缩小了数据扫描范围，提高了查询效率。同时，利用缓存机制将频繁访问的查询结果缓存到内存中，减少磁盘 I/O 操作，并且可以复用结果，进一步提高数据查询效率。文中采用 Spark 缓存机制实现数据湖的缓存，将查询结果存储在内存中，避免频繁读取磁盘的开销，提出了进一步提高查询效率的数据预处理与索引优化技术，对数据进行预处理和创建数据湖中分区数据的索引。最后对提出的方案进行了验证。

中国知网　https://www.cnki.net

第 4 章 数据湖增量计算优化

4.1 引言

数据湖增量计算是指在数据湖中对增量数据进行计算和处理的过程。对数据进行离线批处理和分析，由于数据规模和处理复杂度的增加，批处理方式的处理效率和实时性逐渐降低，已经无法满足实时性和效率的要求[60]。因此，增量计算成为了数据湖中的重要技术之一。增量计算的主要目的是在数据湖中对新到达的数据进行处理，并将处理结果与已有数据进行合并，以得到最新的数据分析结果。增量计算的主要特点包括：1)实时性：增量计算可以在数据到达时立即进行处理，从而实现实时性的数据分析。2)高效性：增量计算只对新增数据进行处理，可以大大减少计算量，提高计算效率。3)可扩展性：增量计算可以通过分布式计算技术实现横向扩展，以应对大规模数据的处理需求。在实际应用中，增量计算主要应用于数据挖掘、实时分析和业务决策等方面。通过增量计算，可以及时发现数据中的异常情况和趋势变化[45]，帮助企业做出更加准确的业务决策。在探索和实践增量数据湖的过程中遇到许多痛点，如时效性、数据集成同步和流批一体的存储介质不统一等问题。针对数据湖的增量计算场景，提出了频繁元素预测再分配算法和分位数合并算法。

本文所提出的数据处理方案采用了 Spark 计算引擎。然而，Spark 在处理数据时通常只进行一次简单的数据划分，因此经常会出现数据倾斜的问题。为解决该问题，一些研究成果提出了精细分区和动态拆分等方法。文献[47,48]中提出精细分区算法通过采样获得近似数据分布生成固定数量的分区，利用拆分函数进行数据重新分区。文献[47]提出 SCID 策略，首先以蓄水池取样法获得近似的数据分布，按数据量大小在 MAP 端对元组进行排序， 对所有分区进行数据迭代填充，如果填充的数据量超过分区容量阈值，为了保证各分区的数据量比较均匀，达到数据合理分配的目的，启动一次拆分过程。SCID 策略需要对所有分区进行重新分布处理，然而在实际应用中通常某几个分区出现分布不均的情况。为了避免对所有分区进行处理，可以提前预测需要处理的分区，进行数据再分配处理。本文提出了频繁元素预测再分配算法（FEPD），该算法可以预测数据中的倾斜元素，并针对倾斜元素进行重新分区，提高数据处理的效率和均衡性。

在大规模分布式数据处理和数据湖场景中，数据的存储是在分布式文件系统中，分布式文件系统最基本存储单位是数据块。文件元数据存储着有关文件的所

中国知网 https://www.cnki.net

有信息，如文件名、文件大小及文件权限等。在大规模数据下，数据块也随之增多，为了提高查询性能需要进行数据块合并。然而数据块合并是一个重要但可能影响性能的操作，尤其是在涉及海量数据汇聚、数据集成和统计分析的时候。传统的数据合并可能涉及到大量的 IO 操作和 CPU 密集型计算，特别是在进行多轮迭代或者合并大量小文件时，会导致性能瓶颈[61-62]。针对这个问题，本文提出了分位数合并算法。

整合两种算法，本文提出了基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案，以提高数据湖的实时性、效率和可扩展性。

4.2Shuffle 机制与数据倾斜

4.2.1Shuffle 机制

Shuffle 是 MapReduce 中连接 Map 任务和 Reduce 任务的桥梁[63]，它负责将 Map 处理后的数据进行随机化分配，并按照分区策略将这些中间数据发送至对应的 Reduce 节点[64]。Spark，作为一个从 MapReduce 数据处理计算框架中汲取灵感的系统[65]，同样整合了 Shuffle 机制。在 Spark 架构内部，Shuffle 过程被精心划分为两个相互衔接的环节，即 Shuffle write 阶段和后续的 Shuffle fetch 阶段。Shuffle write 为下一个计算步骤提供的数据。随后，这些数据根据分区策略被分配到不同的分区桶(bucket)中。这些 bucket 是通过 partition 算法确定的，用于存放 Shuffle MapTask 的输出结果，其存储形式可以是内存、文件或其部分[66]。Spark 中有 Hash Shuffle 和 Sort-Based Shuffle。

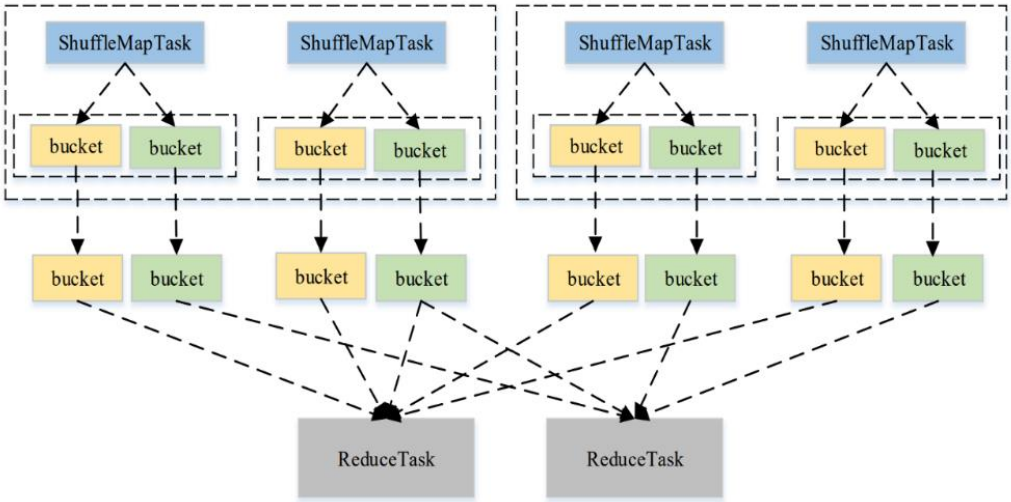

图 4-1　Hash Shuffle 过程

如图 4-1 所示，在 Hash Shuffle 执行流程中，各 bucket 作为由特定分区策

中国知网　https://www.cnki.net

略所指派的目标存储单元，存储来自 Map 任务所产生的输出结果。通常情况下，选取 HashPartitioner 算法来实现数据分割的任务[67]。在此过程中，DAGScheduler 首先接收到 Map 任务产生的 MapOutputTracker-Master 记录，并对其进行转化处理成 MapStatus 形式，这一状态详尽记载了 Reduce 任务在执行拉取操作时所需的关键数据位置及大小属性[68]。接下来，Reduce 任务依据此类精确信息指引，适时地从本地存储或远端节点启动数据拉取流程，从而有效地构造出用于后续处理的 Shuffle RDD 资源[69]。

算法的具体实现，如算法 3 所示：

算法 3：HashPartition 算法

输入：一组键值对数据的 key 元素，分区 num
输出：键值对数据所属的目标分区编号

```
1.if key == null then
2.num = 0;
3.else
4.num = key.hashCode()%numPartitions;
5.end if
```

为克服 Hash Shuffle 存在的潜在问题，Spark 创新性地引入了一种名为 Sort-Based Shuffle 的新颖策略，如图 4-2 所示。该策略的运作背后依托于一个索引文件，用于详尽记录所有 MapTask 的执行输出。通过运用这种技术手段，能够实现对各个文件数据的高效整合，从而有助于缩减内存使用量[68]。相比于传统的 Hash Shuffle 机制，Sort-Based Shuffle 展现出了更为卓越的表现效果。

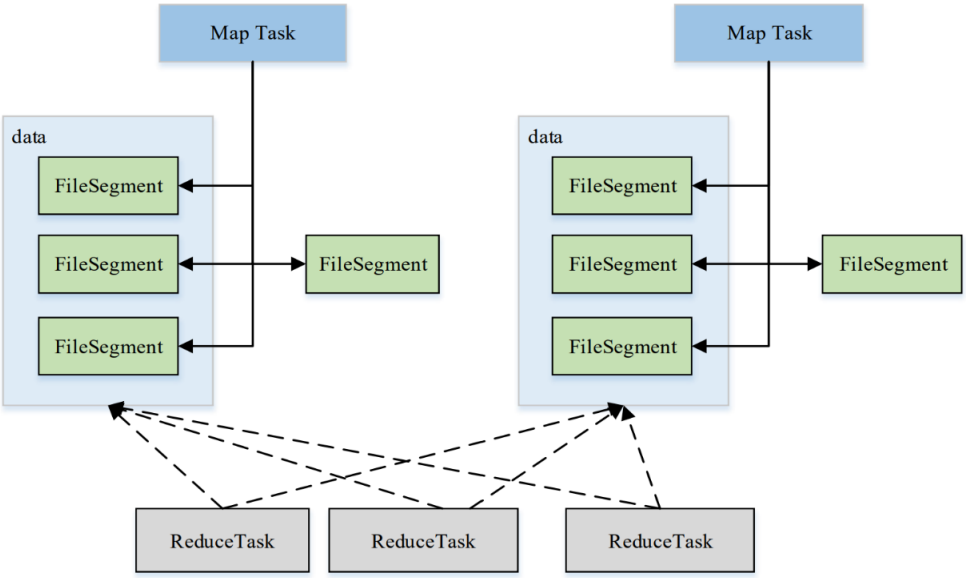

图 4-2　Sort-Based Shuffle 过程

中国知网　https://www.cnki.net

4.2.2Spark shuffle 数据倾斜

在 Hash 分区策略中，拥有相同 key 的中间数据会被分配至同一存储桶中。当某一类具有相同 key 的中间数据构成的数据集合体积极为庞大时，这将引发数据倾斜问题，进而导致对应 Reduce 任务的执行耗时显著增加，严重时甚至可能发生内存溢出现象。考虑到中间数据是以＜key，value＞元组的形式存在，数据倾斜问题主要源于两个维度的原因：1、Key 分布不均：若某一特定类型的 key 元组数目异常庞大，那么由此 key 所引导产生的数据集合尺寸将会增大，进而加剧数据倾斜现象。2、Value 大小不一：在 value 大小可变的情况下，若某种 key 所对应的 value 数据量极其庞大，与其他 key 关联的相对较小 size 的 value 形成鲜明对比，同样会导致严重的数据倾斜问题发生。

4.3 小文件问题分析

当 HDFS 系统中存储的小文件数量急剧攀升时，将对 HDFS 的性能与稳定性构成潜在风险。大量小文件的存在会过度消耗存储资源。由于 NameNode 节点是 HDFS 的具体执行节点，在这种情况下，会增加 NameNode 节点的元数据存储量，即加大系统的内存消耗。从而对 HDFS 系统性能产生显著的负面影响。本章节将选择 HDFS 作为研究背景，系统性地剖析小文件存储问题的根本原因，并进一步阐述 Hadoop 针对大规模小文件存储挑战所提供的现行解决方案。

产生大量小文件的最为常见的原因就是文件划分不合理的。在实际应用场景中，对数据的把控不够，不能客观的对已有数据和未来即将产生的数据进行合理的划分与预测，导致在写入 HDFS 中出现数据分区划分不合理的情况，从而产生了大量的小文件。针对这种情况，小文件的具体阈值成为研究方向之一。为了得出阈值的结论， Dong B 等[55]在 Liu X 的基础上对 0.5～ 64MB 容量大小之间的文件进行了细化划分，取相同文件，小文件判定阈值各不相同的情况下，进行了 23 次合并实验，并通过对每组数据进行大量的测试和分析，最后实验对比多次合并文件所用时间，得出了当小文件的阈值为 4.35MB 时，能够最佳平衡 HDFS 集群节点的存储效率，对于小文件合并具有最佳的性能表现[55]。

在处理和管理大量小型文件时，HDFS 暴露出若干关键性能瓶颈与资源利用效率问题。首要问题源自小型文件的元数据管理特性，由于元数据信息存储于 NameNode 的内存中，每个独立的小文件虽然仅占用一个数据块，却仍会生成固定的 150 字节元数据。当小文件数量激增时，会导致 NameNode 内存被大量占用，同时因每个小文件各自占用一个数据块，使得数据块资源未能得到有效利用。

以存储 1000 万个小文件为例，HDFS 会将这些文件分散存放在 1000 万个独

中国知网 https://www.cnki.net

立数据块中，这无疑加大了元数据存储的压力。另一方面，在访问这些小文件时，客户端与 NameNode 之间的交互变得尤为频繁，包括但不限于：客户端发起请求、NameNode 搜索文件信息、元数据信息传输至客户端、客户端向 DataNode 发送访问请求、DataNode 定位文件位置信息以及将查询结果反馈给客户端等步骤。在大量小文件的访问场景下，频繁的 NameNode 与 DataNode 交互会显著增加 I/O 操作次数，进而导致元数据节点性能下降，访问延迟增加。因此，针对此类问题，亟需寻求策略以减少客户端与 NameNode 之间的通信频率，优化元数据管理，提升 HDFS 在处理大量小文件时的整体性能。

4.4 基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案

基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案是将增量数据处理和 Spark 并行计算技术相结合，以 HDFS 分布式文件系统作为数据存储介质，实现数据湖数据存储方案。主要包括以下四个步骤：1)数据分区;2)数据处理：采用 Spark 并行计算技术对分区后的数据进行处理，包括数据清洗、转换及聚合等操作。Spark 可以通过内存计算和分布式计算技术来提高数据处理的速度和可靠性[70-72];3)数据合并：将处理后的数据合并到原有数据中，以得到最新的数据分析结果;4)数据存储：采用 HDFS 分布式文件系统进行数据存储，将处理后的数据分散存储在不同的节点上[73-75]，以实现数据的并行存储和查询。如图 4-3 所示，本文的数据处理方案包括以上四个步骤。通过数据分区和 Spark 并行计算技术，实现对大规模数据集的高效处理和查询。同时，通过 HDFS 分布式文件系统的数据存储和并行查询，进一步提高数据处理和查询效率。针对 Spark 在 shuffle 阶段出现的数据倾斜问题，提出了一种频繁元素预测再分配算法（FEPD）。而且，针对小文件合并慢问题，提出了分位数合并算法。

图 4-3　基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案流程图

4.4.1 频繁元素预测再分配算法（FEPD）

Spark 框架内的 shuffle 流程实质上是一个将源自 Map 阶段所产生的数据进行重新分配和组织的过程，这一过程旨在将经过初步处理的数据适配到 Reduce 任务的输入规范中，以供后续的 Reduce 阶段进一步聚合和处理。在 Map 过程中，

中国知网　https://www.cnki.net

数据以<key,value>元组的形式作为中间数据，并将 key 值相同的中间数据分配给同一个 Reduce 任务。然而，在实际情况中，若一个 Reduce 任务需处理极为庞大的 key 集合时，往往会遭遇单个任务执行耗时过长的问题，甚至有可能触发内存溢出的现象。

针对上述问题，闫等[20]提出了一种蓄水池采样方法，旨在通过对 shuffle 阶段产生的中间数据进行预测并据此进行分区，以优化数据分布。然而，该策略要求对所有 shuffle 产生的中间数据进行重新分区，对大规模数据，无疑在计算资源和执行效率上提出了较高要求。本文在此基础上进行了拓展与优化，聚焦于解决数据倾斜问题的核心，仅针对那些导致倾斜的 key 值对应的数据元素实施重分区操作，从而规避了对全量数据进行重新分区的高昂成本。在此基础上，本文设计了一种频繁元素预测再分配算法（Frequency Element Prediction and Redistribution, FEPD）。该算法通过三个紧密相连的步骤来提升处理效率和资源利用率：1）key 值预测 2）指定频繁元素分区策略 3）数据重新分区。

1）key 值预测

本文提出了一种基于频繁元素法的 key 值预测方法。该方法可用于对大数据流序列进行 key 值预测。具体而言，从输入的大数据流序列中找出出现最频繁的 k 个 key 值，以进行预测。对大数据序列进行一遍扫描，以确定元素的数量 m,元素种类 n，内存可用单元数 p，从而为预测提供基础。具体实现如算法 4 所示：

算法 4：频繁元素法

```
输入：元素数量 m;元素种类 n;内存可用单元数 p
输出：最频繁的 k 个元素
1 m;
2 n;
3 p[] = 0;
4 for each m in I do
5 if (!p[i] && p[i] == 0) then p[i] = 1; //所属元素计数器不存在且有空闲的计数器，分配计数器并且计数器值+1
end if;
6 if (p[i]) p[i]++;//所有元素计数器存在，计数器值+1
end if
7 if (!p[i] && p != 0 ) then
for each p in j do
p[j] = p[j] – 1 ;//所属元素在计数器中不存在并且所有计数器都不为空，所有计数器的值都-1;
end if;
8 end for
```

完成算法 4 后，可以统计出 k 个出现最频繁的元素:

$$p = \{k_1, k_2, k_3, \ldots, k_p\} \quad (1)$$

中国知网 https://www.cnki.net

其中：p内存可用单元数，k表示数据中的元素

2）制定频繁元素分区策略

根据上一步预测得到的相关数据，需要对数据进行重新分区，因此，本文设计了频繁元素哈希算法，以解决这个问题。具体实现如算法 5 所示：

算法 5 频繁元素哈希算法

```
输入：频繁元素集合 p,分区数量 n
输出：频繁元素分区集合 pt
1 p= {}; //频繁元素集合
2 n;
3 pt={}; //输出元素分区集合
3 for each p I do;
4 pt[i] = hash(p[i])%n;
5 end for;
```

算法 5 的思路是对预测出的频繁元素进行哈希操作，实现数据的重新分区。经过重新分区后，将原始数据的 key 值分散，减少中间数的数量。

4.4.2 分位数合并算法

在分布式文件系统中，查询性能与小文件的存在状态密切相关。在实际使用情景中，首次插入生成的小文件与随后插入形成的同类小文件可能存在内容相似性甚至完全一致的情况，这意味着查询请求可能针对的是重复数据单元，但由于数据分别散落在多个独立的小文件中，查询系统不得不对每一个含有相同数据的小文件进行扫描，从而导致查询效率的显著下降。一旦系统中积累了大量的小文件，这种查询瓶颈效应将会成倍放大，严重影响整个集群的性能表现及其资源利用率。

传统策略在处理小文件聚合问题时，大多仅聚焦于减缩文件数量本身，未能充分考虑写入操作的情境与小文件数量间的交互影响。特别是在那些需要即时写入与即时查询的高动态应用场景下，这种单一维度的处理方式显得尤为局限，无法有效应对数据快速变化与即时访问的双重挑战，凸显了其适用性不足的问题。

为针对性地解决上述问题，本文提出了一种分位数合并算法。该算法设计旨在通过整合考虑数据写入动态特性和小文件的密集程度，优化数据组织与访问机制，从而有效提升在实时数据处理环境下的性能与效率，克服了既往方法在应对高度动态数据流时的不足。

本文强调了小文件问题在分布式文件系统查询性能优化中的核心地位，并指出，随着小文件数量的增长，查询过程中的 I/O 开销与计算复杂度同步上升，这对系统整体性能构成了实质性挑战。因此，探寻有效的策略来规避或减少小文件所带来的查询性能损失，对于提升分布式文件系统在大规模数据分析场景下的表

中国知网 https://www.cnki.net

现尤为关键。如图 4-4，本文要查找一个数据，在分布式文件系统中执行特定标识符(id)的检索操作时，由于缺乏对各小文件内部记录范围的预知信息，检索算法通常无法避免对所有相关小文件进行全面扫描以确保查全率。即使在某些实例中，末尾部分的小文件可能并不涵盖目标查询区间（例如 2 至 4 的数值范围），系统仍然不得不对其进行无差别扫描处理。尤其是在面对大数据环境时，小文件的密集存在极易造成查询效能的大幅下滑，因为频繁的 I/O 操作会导致显著的性能瓶颈。此外，考虑到每次扫描小文件时所伴随的高昂 I/O 成本，当此类小文件数量呈指数级增长时，系统的总体响应时间和资源利用率将受到显著抑制。然而，若采取一种整合策略，即将一系列小文件(此处假设为五个)归并成单一大文件，理论上可以大幅度简化查询流程，仅需在合并后的大文件内执行一次扫描即可定位所需 id，此举有望有效减少不必要的 I/O 操作次数，进而改善系统的整体查询性能。

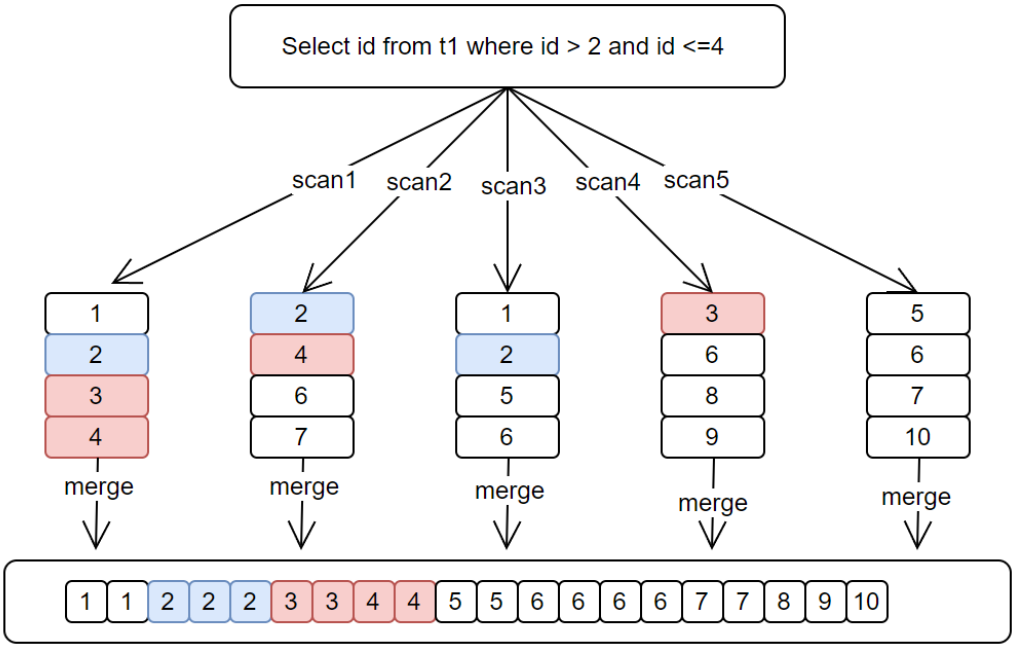

图 4-4　数据查找

在数据合并过程中，资源消耗的关键考量维度涵盖了内存和磁盘两个核心方面。首先，就内存资源而言，由于合并操作通常在内存中执行，故在数据整合期间内存占用呈现上升态势，这是合并过程中内存资源消耗的重要体现。其次，关于磁盘资源，数据合并的本质在于将原先分散存储的数据片段整合至一个新的物理文件中，在合并过程中，原有数据并不会立即被删除，由此引发了所谓的“合并放大”效应，这一效应表现为磁盘空间的暂时性膨胀，通常情况下，合并后磁盘空间的占用增长率介于 1.1 至 1.5 倍之间。此外，合并操作必然伴随着数据的读写活动，从而增加了磁盘 I/O 的负荷，这也是评估合并操作对系统性能影响不

中国知网　https://www.cnki.net

可或缺的因素之一。综合这两方面因素，对合并操作的资源管理与优化策略提出了更高要求，以确保在提高数据存储效率的同时，尽可能降低对系统整体性能的影响。

本文提出的合并策略首先设定一个预设的文件大小标准限值。在 HDFS 文件系统中，常规的数据块大小范围通常设定为 64MB 至 128MB [76-78]。针对待处理数据集中包含的所有文件，本文执行逐一扫描，并据此评估每个文件的大小是否超过了预先设定的阈值。若某个文件的大小超出此阈值，则认为该文件不满足小文件条件，故而可以直接将其存储于 HDFS 中；相反，若文件大小未达到阈值，则认定该文件为小文件，并根据分位数合并算法进行文件合并处理。具体的文件合并操作流程示意以图 4-5 予以详细展示。

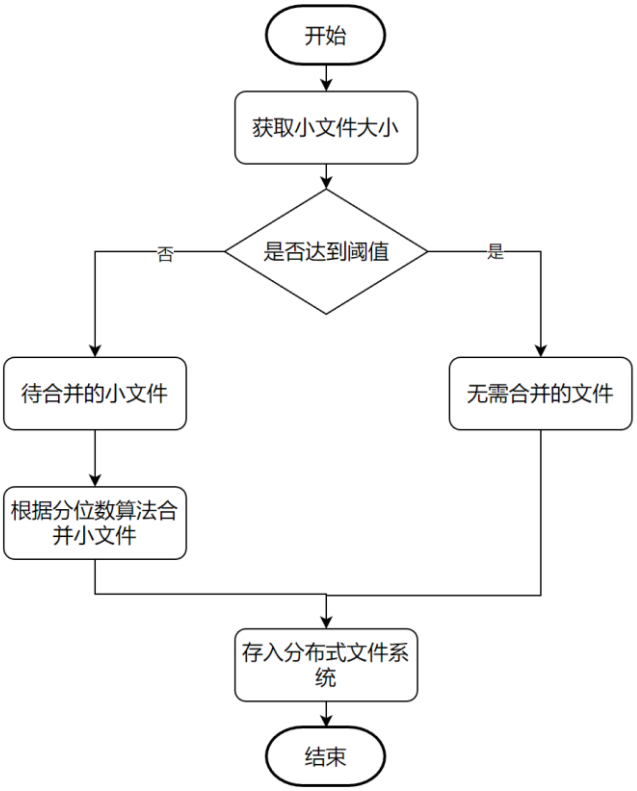

图 4-5　小文件合并流程图

在分布式存储系统中，甄别适宜进行合并的小文件是一项重要的策略决策。对于相邻的两个小文件是否适宜合并，依据以下核心准则进行判断。核心准测如图 4-6 所示：1）避免合并已被锁定并标记为待删除状态的小文件，因其处于生命周期末端，参与合并并无益于长期存储效率的提升。2）排除正在执行合并操作或正处于活跃变更状态的小文件，这类文件由于其数据完整性尚未稳固，强行合并可能导致数据混乱或冲突。反之，对于已完成合并或突变进程且版本状态一致的小文件，则认为它们具备合并的前提条件。综上所述，在确定小文件合并策

中国知网　https://www.cnki.net

略时，首要任务是识别并排除不适合合并的情形，即锁定等待删除的文件以及正在进行合并或发生动态变化的文件。只有当文件状态稳定且版本相符时，才适宜纳入合并流程，以优化存储资源利用并提高系统整体性能。

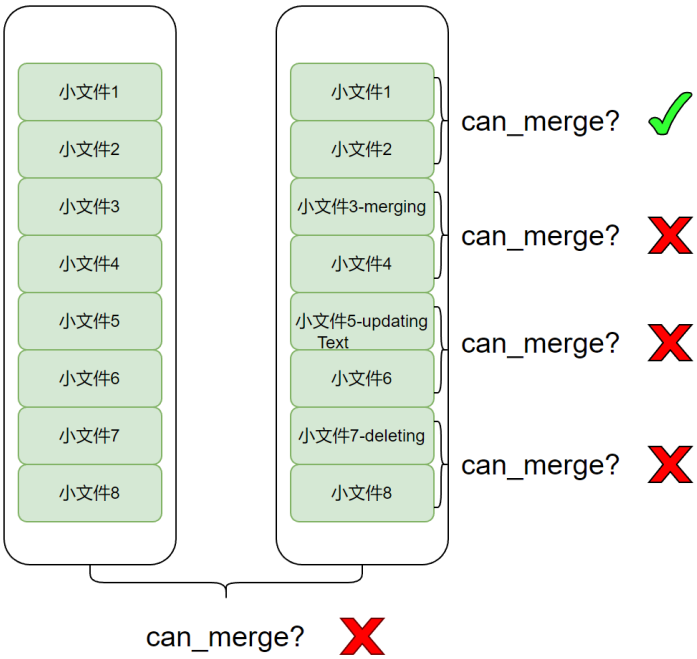

图 4-6　小文件合并判断

分位数合并算法遵循了一系列优化原则，以提高小文件合并效率和集群整体性能：1）关注写入速度：算法会考虑集群的写入速度，避免在合并过程中对写入性能产生负面影响，确保系统在合并小文件的同时仍能维持稳定的写入负载。2）动态监控分位数：通过对集群中小文件大小的实时监控和统计，获取不同大小的小文件分布情况，利用分位数来衡量小文件的分布特性，以此指导合并策略。3）新旧文件合并优先级：考虑到新写入的小文件可能更容易被后续操作访问，因此优先将新文件与其他小文件合并，减少短期内小文件的增长速度。4）大小文件合并策略：由于大文件在合并过程中产生的开销相对较小文件更低，因此相较于小文件，系统会选择优先合并小文件，同时减少对大文件无谓的拆分与合并。5）持续合并存量小文件：即便没有新的小文件写入，系统也应该持续运行后台任务去合并现有的小文件，不断优化存储结构。6）控制小文件数量：由于小文件数量直接影响查询速度，算法力求在任何时刻都将小文件的数量控制在一定范围内，防止查询性能随小文件数量增多而急剧下降。7）并发合并任务管理：虽然可以并发执行多个合并任务以提高效率，但需注意适度控制并发级别，以免过度消耗集群的计算和 I/O 资源，导致整体性能下降。通过以上七条原则，分位数合并算法能在保证系统整体性能的前提下，有效地解决分布式存储系统中的小文

中国知网　https://www.cnki.net

件问题，提高数据湖的存储效率和查询性能。

分位数算法，是为了将多个小文件合并为一个较大小文件，合并过程在内存中执行，这种合并过程可以用树的形式来表示如图 4-7 所示。该算法中有两个重要概念：1）写率：写入磁盘的数据总量相对于原始插入数据量的比率 2）基准值：单次合并过程中所有小文件的总大小与其中最大的小文件的大小的比值，默认为 10。合并的小文件数量是基准值，一般来说基准值设置越大，则合并树就会变得更宽更矮，从而降低写率,可容忍的小文件数量多，基准值越小，则合并越频繁，平均情况下小文件数量也越少。在每次写入小文件大小相近时，基准值应为合并树每个节点的分叉数，合并树的层高大致为小文件数关于基准值的对数。基准值这个值变化的原则如下：1）要合并的小文件总大小过大时减少基准值 2）要合并小文件时间过长时减少基准值 3）分区内的小文件的数量过多时减少基准值。如算法 6 所示：

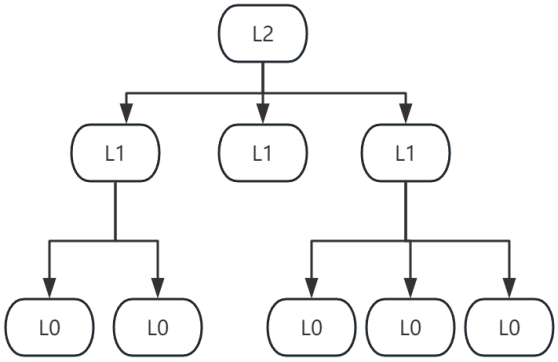

图 4-7　合并树结构图

算法 6：分位数算法

输入：写入数据量 n;小文件总量 m;每次合并最大数 max_num；
输出：文件数量 m;基准值 b;

```
1 n;//写入数据量
2 m;//小文件总量
4 max_num;//每次合并最大数
5 dm = get();//获取磁盘存储量
6 v = dm / n;//计算写率
7 max_size;//小文件中最大的文件
8 for each m in i
   if(m[i]>max_size) max_size = m[i];
   end;//查找最大的文件
9 b = sum(m) / max_size;//计算基准值
10 for each m in j
   merge(m[j])
   end;//合并小文件
```


中国知网　https://www.cnki.net

4.5 实验与分析

4.5.1 频繁元素预测再分配算法实验

基于上述数据集，本文设计了两种不同分区策略效率的实验。实验 1 按照 uuid 字段进行分区，采用了预测算法预先识别 key 值分布，随后对全部数据实施全面的重新分区处理，以期达到数据均衡的目的，并记录完成此过程所需的写入时间。实验 2 按照 uuid 字段进行分区，利用 key 值预测算法，但仅针对预测中识别出频率高的 key 值对应的数据执行重新分区操作，旨在通过针对性优化来减少不必要的数据移动，同时记录相应的写入时间。为确保实验结果的可靠性和稳定性，每种实验均重复执行三次，并计算所有测量数据的平均值，以此作为评估两种策略效果的基准。这一研究设计不仅考察了预测算法在数据预处理中的应用效果，还通过对比分析两种分区策略对写入效率的影响，进一步探讨了在大数据处理场景下优化资源分配与提升处理速度的有效途径。

根据表 8 所示的实验数据结果，实验 1 在执行全量数据重新分区后的平均写入时间为 310.6 秒，而采取了针对性重新分区策略的实验 2，其平均写入时间则显著减少至 210.3 秒。对比数据结果，表明了基于预测算法且仅针对高频 key 值数据进行重新分区的策略在提高写入效率方面的优势。

表 8 缓存机制实验结果

实验名称	写入时间/s	平均值/s
实验 1	321	310.6
	299	
	312	
实验 2	210	210.3
	211	
	210	

4.5.2 分位数合并算法实验

基于上述选定的数据集，本文部署了两项实验来深入探究不同策略对实时写入与实时查询效率的影响。实验 1 着重于传统的处理方法，即在不特别强调写入速率的前提下，集中关注减少小文件的数量，通过执行小文件合并操作。在此实验中，于特定观测时段内，系统地记录了写入操作与查询响应的总时间，以此来评估单纯基于小文件数量减少的策略效果。实验 2 采用分位数合并算法，在实时数据交互的场景下进行，旨在通过更智能化的小文件合并策略来优化数据管理。该实验同样在设定的时间段里，监测并记录了数据写入与查询操作的所需时间，旨在展示算法在实际应用中的性能提升，尤其是在平衡写入效率与查询响应时间方面的能力。

中国知网　https://www.cnki.net

根据图 4-8 所示的实时写入性能分析结果，可以观察到两个实验截然不同的趋势特征。具体而言，实验 1 在持续的实时数据写入过程中，表现出写入性能逐步下降的趋势。相比之下，实验 2 在整个观测周期内，其写入性能维持在一个相对稳定的水平，未显示出明显的性能衰退迹象，表明所采用的分位数合并算法有效保障了在实时写入场景下的性能一致性与稳定性。

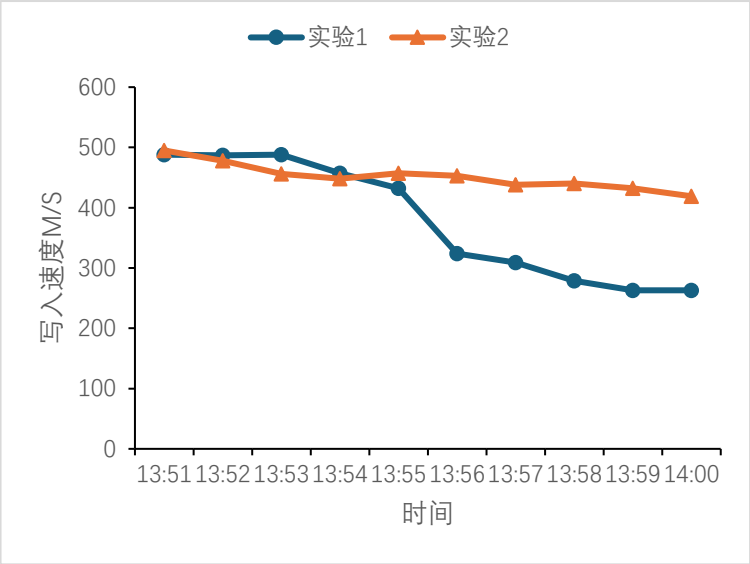

图 4-8　实时写入对比图

根据图 4-9 所示的实时查询性能评估，可以清晰地辨析出两个实验在查询耗时上的差异化表现。在实验 1 中，随着查询操作的持续进行，查询响应时间呈现出了明显的上升趋势。实验 2 则展示出了一种更为理想的态势，其查询耗时在经历初期变化后逐渐趋向于稳定状态，表明所采用的策略有效缓解了查询性能随时间推移的衰减问题，保持了较高的查询效率与系统稳定性。这一对比结果深刻体现了分位数合并算法在优化实时查询场景下性能稳定性方面的显著优势。

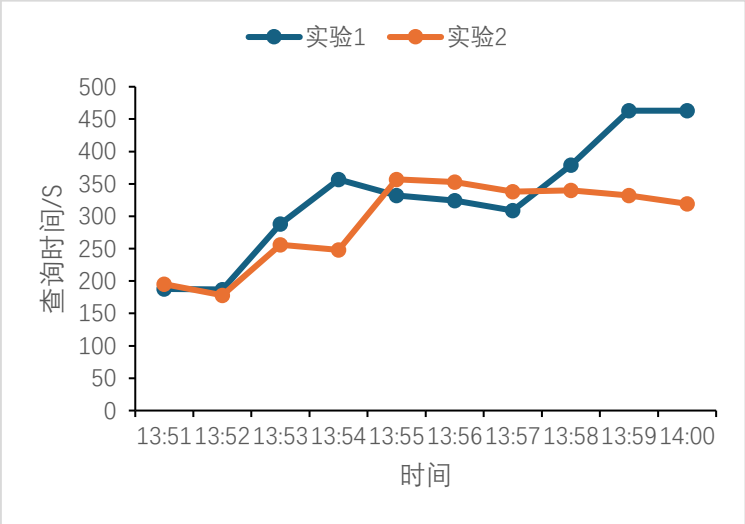

图 4-9　实时查询对比图

中国知网　https://www.cnki.net

4.5.3 数据湖增量计算优化实验

本小节根据第三章中的实验环境和数据集进行了两个实验，模拟了数据湖增量计算的场景。实验 1 采用直接写入的方式，先将第一个文件按照 uuid 的进行分区并写入 HDFS，再将剩下的 10 个文件依次写入 HDFS，并对第一个文件的数据进行更新操作，统计写入的总时间。写入完成后对数据进行查询操作，并统计查询总时间。实验 2 采用频繁元素预测再分配算法（FEPD），对第一个文件按照 uuid 的值进行统计，一共有 100 种 uuid，其中五种 uuid 的数量远大于其它 uuid 的数量，采样结果如图 4-10 所示。对这五种 uuid 进取模 100 操作，进行重新分区。采用 Spark 写入 HDFS 操作，统计写入时间。通过对 HDFS 文件的遍历，根据分位数合并算法按照阈值 64MB 合并小文件操作后，对数据进行查询操作，并统计查询总时间。针对这两种实验分别进行三次实验，实验结果如表 9。

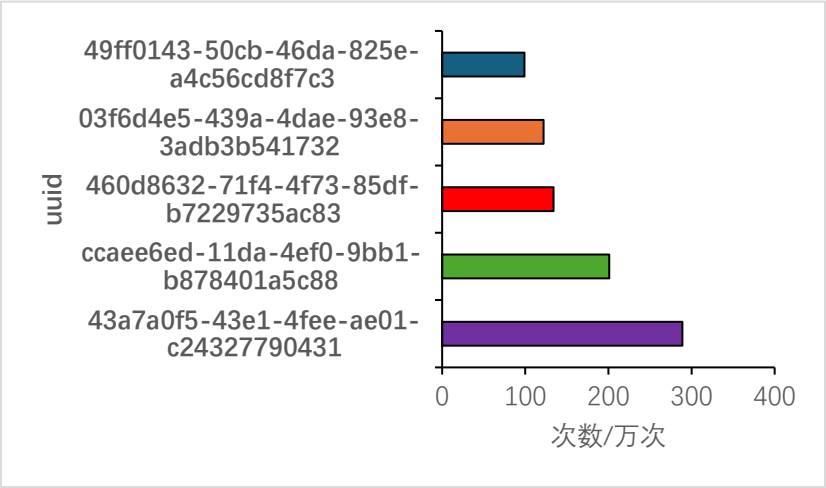

图 4-10　uuid 预测图

通过对比实验结果。在数据湖增量计算的场景下，直接写入的时间 612.3 秒，直接查询的时间 154.6 秒。然而，基于 Spark 并行计算和 HDFS 分布式存储模型在本文实验中写入时间 210.3 秒，读取时间 70.6 秒。通过对比数据湖的存取性能有比较明显的提高。

表 9 基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案实验结果

实验名称	写入时间/s	平均时间/s	查询时间/s	平均时间/s
实验 1	612	612.3	154	154.6
	613		155	
	612		155	
实验 2	210	210.3	70	70.6
	211		71	
	210		71	

中国知网　https://www.cnki.net

4.6 本章小结

本章节针对数据湖增量计算场景提出了基于 Spark 并行计算和 HDFS 分布式存储的数据处理方案，并对 Spark Shuffle 机制进行了分析，探讨了 Spark shuffle 数据倾斜的原因和影响。提出了一种预测方法，动态调整 Reduce 任务分配策略，实现负载均衡，从而降低数据倾斜的影响。同时，对小文件问题进行了分析，提出了一种基于数据大小的文件合并策略，根据分位数合并算法将小文件合并成较大的文件，以提高数据湖的读取性能。最后对提出的方案进行了实验。

中国知网　https://www.cnki.net

第 5 章 基于 Hudi 的数据湖系统实现

本文旨在利用 Spark 技术、HDFS 技术和 Hudi 技术研究数据湖存取性能优化方面内容，最后构建数据湖系统。本章将阐述数据湖系统的功能。

5.1 需求场景分析

本文聚焦于满足商业化报表需求的典型在线分析处理（OLAP）场景，其中数据源自关系型数据库系统，且该数据集普遍经历频繁的更新操作，数据一共有 1324 万条。在数据迁移与处理流程中，规定从源数据库提取数据并将其加载至 Apache Hudi 存储系统的延迟不得超过 1 小时，以确保数据新鲜度。一旦数据成功写入 Hudi，进一步要求执行数据聚合操作以满足复杂多维分析需求。针对最终查询性能，研究设定明确的服务质量标准：在单次 SQL 查询执行过程中，响应时间必须控制在 5 秒以内，并在高并发条件下，系统应能支撑每秒至少处理 500 次查询请求（QPS 达到 500）。该场景下一共有 12 个数据字段如表 10 所示，其中广告 id、广告组 id、推广应用包 id 和时间为常用的聚合维度字段。

表 10 商业化报表数据字段以及类型

序号	字段名称	字段类型	注释
1	time	int	报表统计时间
2	advertisement_id	string	广告 id
3	group_id	string	广告组 id
4	app_id	string	推广应用包 id
5	app_package	string	推广应用包名
6	company	string	所属公司
7	show_count	int	显示次数
8	clickh_count	int	点击次数
9	download_count	int	下载次数
10	register_count	int	注册量
11	retain_ratio	decimal(14,4)	留存率
12	rechage_amount	decimal(14,4)	充值金额
13	day	date	日期

中国知网 https://www.cnki.net

5.2 技术架构选型

在进行技术栈抉择时，本文重点关注了 Iceberg 与 Hudi 这两个开源组件的功能特性比较。从表 11 可以观察到两者在若干关键功能上的异同。值得注意的是，Hudi 所展现出的实时更新特性、强大的数据更新能力、遵循 ACID（原子性、一致性、隔离性和持久性）事务特性和对数据有序性的维护，在构建满足业务场景需求的端到端实时数据处理解决方案中，被识别为至关重要的几个核心特征要素。这些特性使得 Hudi 在处理实时数据流及确保数据准确性和完整性方面体现出显著的优势。

实时性方面，数据表的读写性能对整个数据加工处理性能具有决定性影响。特别是在采用了 Hudi 的多版本优化读写（MOR）机制之后，表的读写性能得到了显著提升，尤其增强了对增量数据的实时处理能力。

更新能力是衡量数据湖方案有效性的重要指标。传统的数据湖实现方法在处理数据更新时复杂度较高，通常需要构建拉链表并通过关联查询发现待删除和更新的数据项，这一过程的时间复杂度与表的规模紧密相关，往往导致从数据入湖到处理完成的整体性能较差。相比之下，支持原生数据更新能力的方案能够显著优化此环节的性能表现。

ACID（原子性、一致性、隔离性和持久性）属性对于提升系统性能和数据完整性至关重要。在缺乏 ACID 保证的传统多任务执行环境中，任务之间存在着严格的依赖关系，上层任务完成之前，下层任务无法启动，这导致了内部业务间乃至跨业务作业调度关系的复杂性加剧，进而影响了端到端的整体性能表现。然而，具备 ACID 能力的系统能够确保读取的数据始终是已成功写入的完整状态，从而改变原有的调度策略。不再依赖于 Pipeline 的严格序列化执行，而是允许上游数据一旦生成，下游即可立即进行处理，进而将上下游作业从串行执行转为并行执行，显著提升了整体执行效率。

至于数据保序问题，在数据写入流程中，尤其是采用变更数据捕获（CDC）技术收集业务数据库数据时，数据的时序性是保证数据准确性的基石。一旦数据写入的顺序遭到破坏，数据的正确性将难以得到保障。此外，当数据源业务系统遭遇异常并重新发送数据时，若缺乏数据保序机制，则计算结果的准确性也无法得到有效保证。因此，确保数据写入过程中的时序性对于系统整体性能和数据正确性具有不可忽视的重要性。

从功能上来看 Hudi 更能满足项目要求。同时从后续演进来看，Hudi 的社区活跃度更高，基于可以持续演进的考虑，将 Hudi 做为底层的架构组件。

中国知网 https://www.cnki.net

表 11 两种数据湖功能特性对比

功能特性	Iceberg	Hudi
ACID	支持	支持
数据保序	不支持	支持
实时性	偏离线	准实时
数据维护（小文件、历史版本清理等）	用户高代码解决	自动解决
更新效率	低	高
更新能力	全行更新	支持部分更新、全行更新
离线处理	支持	支持
隐式分区	支持	不支持
分区演进	支持	不支持

5.3 系统架构设计

本文的性能测试实验是在一套综合集成多种核心技术组件的数据湖系统上展开的，该系统构建了一套高效、可扩展的数据加工与分析生态环境如图 5-1 所示。该数据湖系统的基础架构主要由三大支柱构成，即 Spark 计算集群、HDFS 分布式文件系统以及 Hudi 数据湖。这三个核心组件紧密协作，为用户提供强大的数据处理潜能和灵活的数据存储策略。

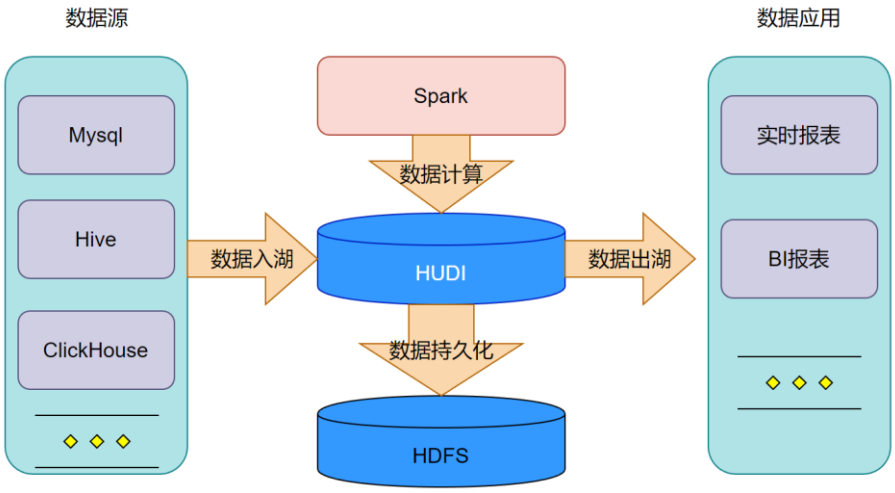

图 5-1　系统架构图

Spark 集群作为该数据湖系统的核心计算动力源，擅长高效驾驭大规模数据集，并借力于其优越的并行计算性能，实现对数据的快速分析与处理。HDFS 分布式文件系统则负责提供稳健且高效率的数据存储机制，确保数据在扩容和容错方面的优良性能。而 Hudi 数据湖凭借其独到的增量数据处理优势，为数据湖系统

中国知网　https://www.cnki.net

带来了更为灵动且高效的更新与管理机制。

此外，数据湖系统还配备了完善的用户界面功能集合，涵盖数据库管理、数据表管理及交互查询功能模块，使得用户能够更加便捷地进行数据管理工作，从而实现对数据资源的全面掌握和高效利用。

5.4 系统硬件环境与环境搭建

Spark 运行平台：Spark 计算层设置 3 节点，其中 1 个主节点，32GB 内存，32 个 cpu 内核（Intel i5-8500 处理器），2 个工作节点，每个工作节点 32GB 内存，16 个 cpu 内核（Intel i5-8500 处理器）。HDFS 分布式文件系统：HDFS 存储层设 3 个节点，这 3 个节点均为 32GB 内存，16 个 cpu 内核（Intel i5-8500 处理器），其中 1 个主节点，2 个数据节点，。Hudi 数据湖：4 个节点，这 4 个节点均为 32GB 内存，16 个 cpu 内核（Intel i5-8500 处理器）。

数据湖系统搭建分为三个部分:Spark 集群搭建、HDFS 环境搭建、Hudi 搭建。Spark 集群用来对数据的计算以及数据写入数据湖，HDFS 环境用来存储数据，Hudi 用来对数据进行管理。

5.5 应用场景接入

应用场景通过系统功能模块接入，接入流程如图 5-2 所示，具体内容概述如下：原始数据首先通过 Spark 分布式计算框架进行处理，并以此作为中介将数据高效地写入 Hudi 数据湖存储系统中，随后，这些经 Hudi 规范化管理的数据得以永久性存储在 HDFS 之上，实现了数据的可靠持久化存储，并进行报表分析。

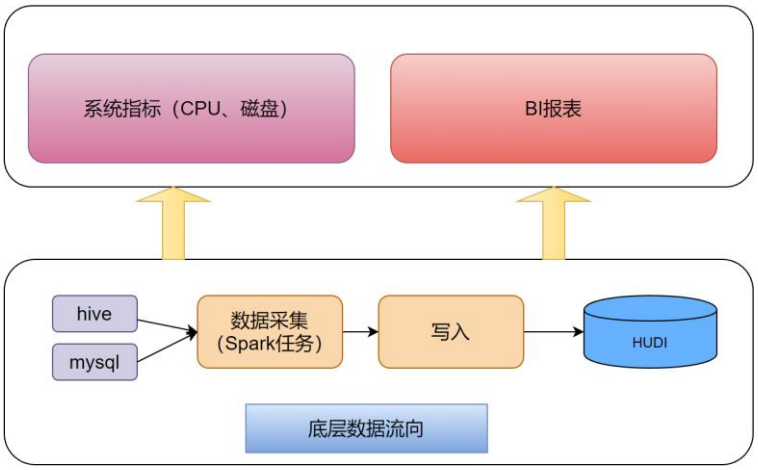

图 5-2 应用场景接入

中国知网 https://www.cnki.net

系统被划分为三个相互关联的功能模块：一是数据库管理体系，二是数据表管理模块，三是交互查询机制。

首先，数据库管理体系主要用于构建和管理 Hudi 数据库。具体而言，通过该模块可以实现 Hudi 数据库的创建操作，如图 5-3 所示，展示了如何通过数据库管理功能模块新建一个 Hudi 数据库实例的过程。

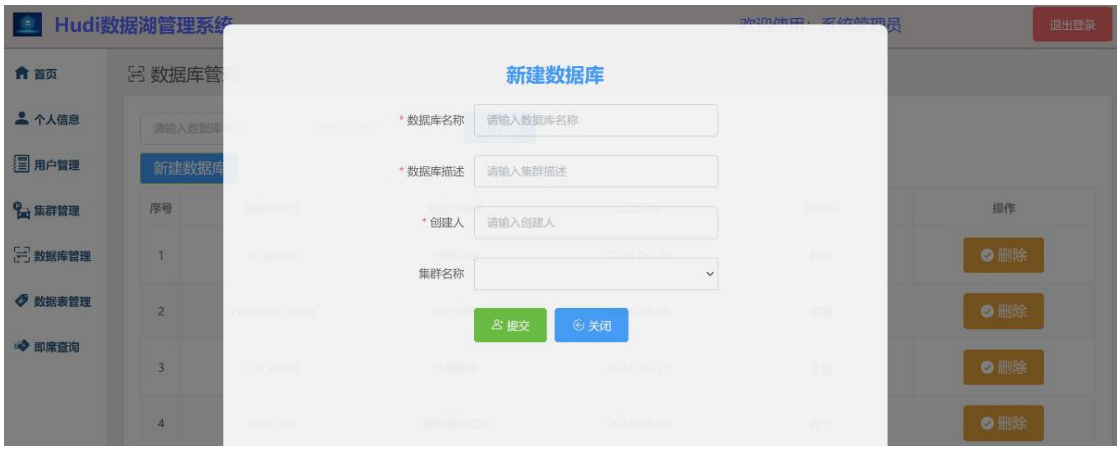

图 5-3　新增 Hudi 数据库

借助于 Hudi 数据表管理模块，可以实现新增 Hudi 表的操作，这一过程的具体演示如图 5-4 所示。该模块允许用户创建新的 Hudi 表结构，以便进行数据存储和管理，从而扩展了系统的数据处理能力。在该模块中，用户可以定义新表的字段、分区策略，确保新增的 Hudi 表能够满足特定应用场景的需求，并与现有 Hudi 数据库架构无缝对接。

图 5-4　新建 Hudi 表

借助于交互式查询界面，用户能够有效地检索和探究 Hudi 中的数据内容，如图 5-5 所示。该界面设计旨在提供一种直观且用户友好的方式，使得研究者能够灵活地查询和分析存储在 Hudi 系统中的各类数据信息。通过输入特定查询条

中国知网　https://www.cnki.net

件，用户能够在该界面上实时获取所需结果，从而实现对 Hudi 数据集的深度洞察与高效利用。

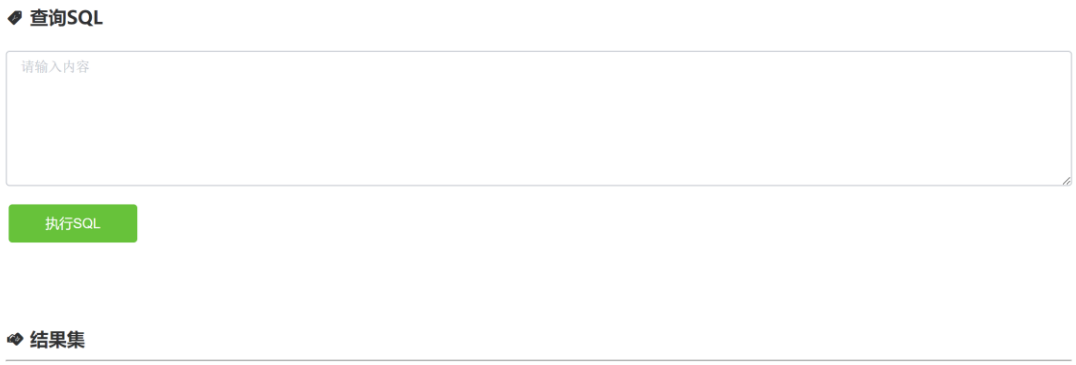

图 5-5　交互查询

5.6 性能分析与优化

在将商业化报表业务接入数据湖系统后，观察到其无法满足关键性能指标，即查询响应时间需控制在 5 秒以内以及每秒查询量(Queries Per Second, QPS）达到 500 的要求。对此现象进行了深入的性能评估研究，主要聚焦于三个核心优化方向：首先，对数据分区策略的有效性进行了详尽审查，探究其是否能够均衡且高效地分布数据以加速检索；其次，分析了在数据计算流程中是否存在数据倾斜问题，此问题可能导致部分计算资源过度使用，进而影响整体系统的响应速度；最后，对小文件合并操作的效率进行了细致检验，旨在明确该环节是否成为制约系统整体查询性能的关键瓶颈。

在业务接入初期阶段，针对研究场景采用了以 day 字段为基础的数据分区策略，然而，实际观测到的现象表明，数据在各个节点之间的分布呈现出明显的不均衡性，这一状况在查询操作执行过程中尤为突出，表现为节点间 CPU 利用率的显著差异，具体如图 5-6 所示。此外，数据集中的聚合操作占据相当高的比例，致使查询响应时间延长至无法满足实际业务需求的程度。为解决这一问题，本文引用了基于数据分区与缓存机制相结合的访问优化方案，对该场景下的数据分布进行了深入剖析。统计分析结果显示，聚合操作所依据的广告 id 字段表现出分布不均匀的特点，详情如图 5-7 所示。据此，本文进一步采纳了文中提出的哈希算法策略，对数据分布进行了重新调整，确保数据在各个节点间更为均匀地分布。同时，将预处理后的聚合数据直接存储于数据湖中，以优化查询过程。通过上述优化措施的实施，查询性能得到了显著提升。

中国知网　https://www.cnki.net

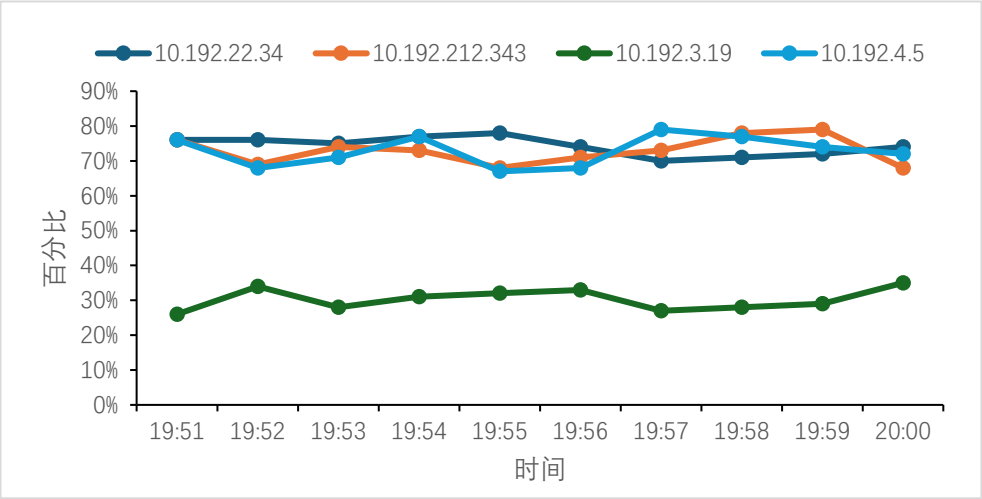

图 5-6　CPU 空闲率图

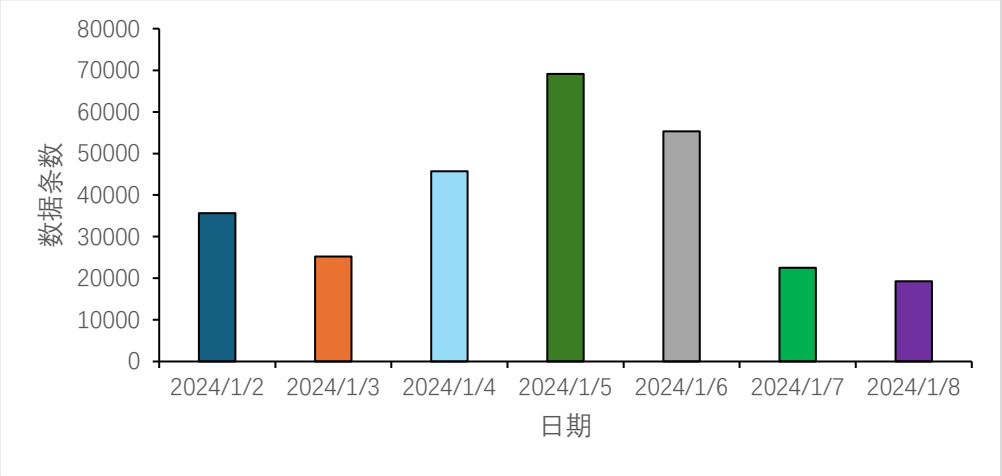

图 5-7　广告 id 字段数据分布图

在本文中，数据来源多样性，涉及多个关系型数据库，采用 Spark 分布式计算框架对原始数据进行了预处理。处理统计结果如图 5-8 所示，揭示了在依据广告 id 和 group_id 两个字段进行数据聚合过程中存在的显著数据倾斜现象。针对这一问题，应用了频繁元素预测再分配算法进行了针对性处理，有效地缓解了数据倾斜问题，提升了数据处理的均衡性。然而，在进一步利用预处理后数据进行查询分析时，发现查询性能仍未能达到预期需求。因此，对数据合并过程进行了详尽分析，结果显示在数据合并阶段，内存资源的使用率极高，如所图 5-9 所示，这种资源消耗状况显著制约了查询性能的表现。为解决此瓶颈，本文实施了提出的分位数合并算法，通过该算法优化了数据合并流程，有效地降低了 CPU 和内存的使用率，从而显著提升了系统的查询性能。

中国知网　https://www.cnki.net

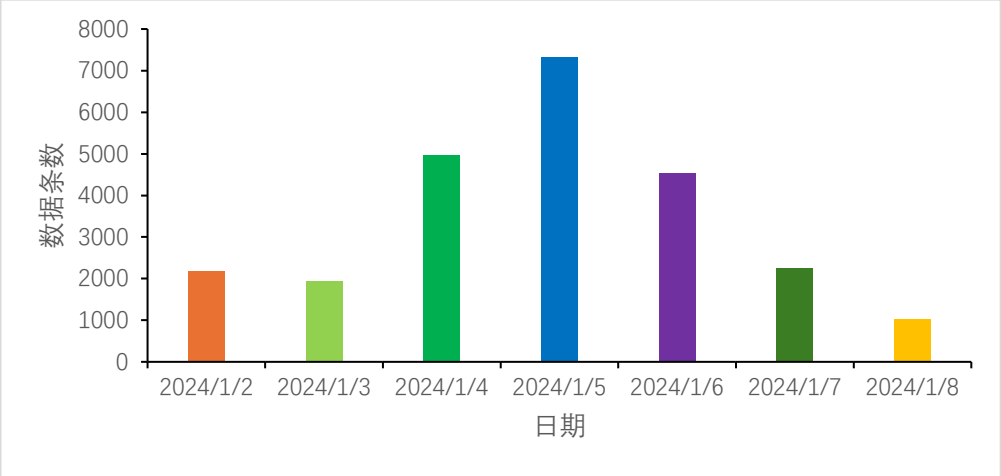

图 5-8 数据聚合结果统计图

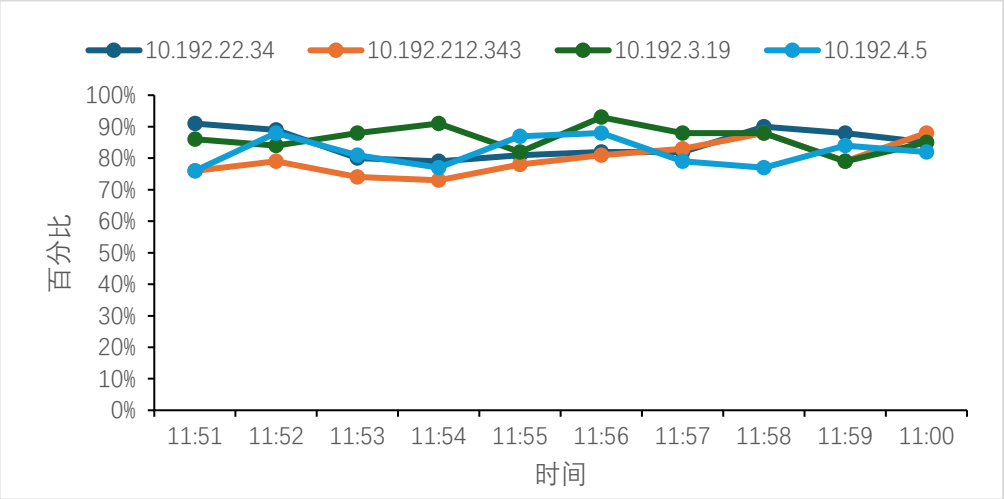

图 5-9　内存利用率图

本文深入探讨了如何通过先进的数据湖技术和优化策略来改进系统的数据写入性能、查询效率以及并发处理能力。首先，本文详细研究了如何借助批处理和分区技术优化小时任务和天任务的数据写入流程，以减少 I/O 操作次数并提高数据写入的并行度。其次，本文探索了数据预处理和预聚合策略在满足求和分析需求方面的潜力，以及如何设计和实施高效的数据模型以支持多维度查询。此外，本文还研究了如何运用列式存储、索引技术和查询优化算法来缩减查询耗时，并结合动态资源调度和集群扩展策略来保证在高 QPS 条件下的查询性能。在本文中，采用 apiPost7 工具实施了若干轮次的压力测试，依据所得实验数据可确在性能要求方面达到了预期标准。测试结果如图 5-10 所示：

中国知网　https://www.cnki.net

图 5-10　测试结果图

5.7 本章小结

本章节详细论述了数据湖系统构建的底层硬件配置环境，以及大数据组件在数据湖平台架构中的具体实现与集成。进一步，本文结合了商业化报表的实际应用场景，对其进行了深度分析并实施了相应的优化策略。在此基础上，本文详尽地展示了数据湖系统的构建流程及其核心功能模块，并对数据湖系统的用户界面展现功能做了具体的阐述。通过上述探讨，全面描绘了数据湖系统的整体构建框架及其在实践应用中的功能特性。

中国知网　https://www.cnki.net

第 6 章 全文总结与展望

6.1 全文总结

本文旨在探讨数据湖存储性能优化的相关技术、以提高数据湖的存储、访问及处理性能。具体的研究工作包括以下几个方面：

首先，本文对数据湖存储结构的各种类型及其特性进行了详尽的概览和阐述。其次，本文提出了一个创新的访问优化策略，该策略基于数据分区技术和缓存机制，旨在改善数据湖的访问效率和响应时间。随后，本文进一步探讨了一种结合 Spark 并行计算框架与 HDFS 分布式存储技术的数据处理方案。通过此项研究，本文证实了合理设计的数据分区策略、精细优化的索引技术以及高效的缓存机制在提升数据存储效率和查询性能方面具有显著成效。同时，采用 Spark 并行计算与 HDFS 分布式存储技术相结合的方案，能够显著增强数据湖对增量计算任务的支持能力，进而满足实时数据分析和业务决策的需求。然而，实际应用中，须根据不同数据集的内在特性和具体使用场景的差异，审慎选择和定制最适宜的技术方案，以确保实现最佳的数据处理效能。总结而言，本文为数据湖存储系统的性能优化贡献了新的理论思考与实践路径，有力地促进了数据湖技术在各类应用场景下的高效利用，从而增强了数据分析和业务决策过程中的数据处理能力。

6.2 后续工作展望

在本文实验阶段，所设定的查询情境表现为一种简化且特定的形式，并未涉及对不同查询条件下算法时间复杂度的具体考量。未来的研究迭代中，计划纳入多元查询场景以进行性能对比分析，并将开展针对算法在不同场景下时间复杂度的深入探究，旨在全面评估其在多种实际应用环境中的效率表现。

中国知网 https://www.cnki.net

参考文献

[1] 夏立斌,刘晓宇,姜晓巍,等.基于分布式数据集的并行计算框架内存优化方法[J].计算机工程,2023,49(04):43-51.

[2] 赵丽梅,黄小菊,宫学庆.Spark查询引擎中Join操作的优化[J].计算机应用与软件,2022,39(08):44-50.

[3] Ma C ,Zhao M ,Zhao Y .An overview of Hadoop applications in transportation big data[J].Journal of Traffic and Transportation Engineering(English Edition),2023,10(05):900-917.

[4] Li H ,Ji S, Zhong H, et al. LPW: an efficient data-aware cache replacement strategy for Apache Spark[J].Science China(Information Sciences),2023,66(01):77-96.

[5] Huang S, Xu J, Liu R, et al. A novel compression algorithm decision method for spark shuffle process[C]//2017 IEEE International Conference on Big Data (Big Data). IEEE, 2017: 2931-2940.

[6] Wang J , Qiu M , Guo B ,et al. Phase–Reconfigurable Shuffle Optimization for Hadoop MapReduce[J].IEEE Transactions on Cloud Computing, 2015, PP(99):1-1.

[7] Shang T ,Zhao Z ,Ren X , et al. Differential identifiability clustering algorithms for big data analysis[J].Science China(Information Sciences),2021,64(05):49-66.

[8] Yanfei G ,Jia R, Dazhao C, et al. iShuffle: Improving Hadoop Performance with Shuffle-on-Write[J].IEEE Transactions on Parallel and Distributed Systems,2017,28(6):1649-1662.

[9] Shuai G ,Meng S ,Xiaodong M .A Metadata Reconstruction Algorithm Based on Heterogeneous Sensor Data for Marine Observations[J].Journal of Ocean University of China,2023,22(06):1541-1550.

[10] Sandhu K A .Big Data with Cloud Computing :Discussions and Challenges[J].Big Data Mining and Analytics,2022,5(01):32-40.

[11] Baruah J A ,Baruah S .Data Augmentation and Deep Neuro-fuzzy Network for Student Performance Prediction with MapReduce Framework[J].International Journal of Automation and Computing,2021,18(06):981-992.

中国知网 https://www.cnki.net

[12] Satish C ,P. V ,Praveen D .A Parallel Fractional Lion Algorithm for Data Clustering Based on MapReduce Cluster Framework[J].International Journal on Semantic Web and Information Systems (IJSWIS),2022,18(1):1-25.

[13] Yang D ,Cheng D ,Rang W ,et al .Joint Optimization of MapReduce Scheduling and Network Policy in Hierarchical Data Centers[J].IEEE Transactions on Cloud Computing, 2019, PP(99):1-1.DOI:10.1109/TCC.2019.296165

[14] Wu X D , Ji S W .Comparative Study on MapReduce and Spark for Big Data Analytics[J].Journal of Software, 2018.DOI:10.13328/j.cnki.jos.005557.

[15] Powers ,J ,Chen ,et al .CRESP: Towards Optimal Resource Provisioning for MapReduce Computing in Public Clouds[J].IEEE Transactions on Parallel and Distributed Systems: A Publication of the IEEE Computer Society, 2014, 25(6):1403-1412.

[16] Nti K I ,Quarcoo A J ,Aning J , et al .A Mini-Review of Machine Learning in Big Data Analytics: Applications ,Challenges ,and Prospects[J].Big Data Mining and Analytics,2022,5(02):81-97.

[17] Bei Z ,0001 Y Z ,Zhang H , et al. RFHOC: A Random-Forest Approach to Auto-Tuning Hadoop's Configuration.[J].IEEE Trans. Parallel Distribute. Syst.,2016,27(5):1470-1483.

[18] Fang H .Managing data lakes in bigdata era :what’s a data lake and why has it became popular in data management ecosystem[C]//IEEE.Proceedings ofthe2015IEEE International Conference on Cyber Technology in Automation ,Control ,and Intelligent Systems.Piscataway:IEEE,2015:820-824.

[19] Li F ,Yu X ,Ge R , et al .BCSE: Blockchain-Based Trusted Service Evaluation Model over Big Data[J].Big Data Mining and Analytics,2022,5(01):1-14.

[20] 阎逸飞, 王智立, 邱雪松, 王嘉潞. Spark环境下基于数据倾斜模型的Shuffle分区优化方案[J]. 北京邮电大学学报, 2020, 43(02): 116-121.

[21] 陈氢,张治.融合多源异构数据治理的数据湖架构研究[J].情报杂志,2022,41(5):139-145.

[22] 陈氢,宋仕伟.数据治理视角下的湖仓一体架构研究[J].数字图书馆论坛,2023,19(04):19-28.

[23] 黄学雨,向驰,陶涛.基于MapReduce和改进密度峰值的划分聚类算法[J].计算机应用研究,2021,38(10):2988-2993+3024.

[24] Khing P P, Wang ZS. Data lake: a new ideology in big data era[C]//Editi

中国知网 https://www.cnki.net

on Diffusion Presse Sciences. Proceedings of the 4th Annual International Conference on Wireless Communication and SensorNetwork.Paris:EDPSciences,2018:1-11.

[25] Beheshtia, Benatallah B, Nourir , et al. CoreDB: a data lake service[C]//ACM. Proceedings of the 2017 ACM on Conference on Informationand Knowledge Management.NewYork:ACM,2017:2451-2454.

[26] Hu Z ,Li D .Improved Heuristic Job Scheduling Method to Enhance Throughput for Big Data Analytics[J].Tsinghua Science and Technology,2022,27(02):344-357.

[27] Zagan E, Danubianu M. Cloud DATA LAKE: the new trend of data storage[C]//IEEE. Proceedings of the 2021 3rd International Congress on Human-Computer Interaction, Optimization and Robotic Applications. Piscat away: IEEE,2021:1-4.

[28] 张宁,袁勤俭.数据治理研究述评[J].情报杂志,2017,36(5):129-134,163

[29] 周翔,翟俊海,黄雅婕,等.大数据环境下的投票特征选择算法[J].小型微型计算机系统,2022,43(05):936-942.

[30] Derakhshannia M, Gervetc, Hajj-Hassanh, et al. Data lake governance: towards a systemic and natural ecosystem analogy[J].FutureInternet,2020,12(8):126-135

[31] Brousp, Janssen M, Kransr. Data Governance as Success Factor for Data Science [M].Switzerland:Springer,2020:431-442.

[32] 周晴红．Hadoop 大数据开发技术课程实践教学[J]．办公自动化，2021，48:177-182．

[33] 杜鹃,张卓,曹建春.利用快速无偏分层图抽样算法的MapReduce负载平衡方法[J].计算机应用与软件,2021,38(11):288-294+313.

[34] 杨文哲,郝渊科,赵常胜,等.基于对象代理的大数据共享可信数据湖平台[J].小型微型计算机系统,2023,44(06):1324-1328.DOI:10.20009/j.cnki.21-1106/TP.2021-0753.

[35] 杨良怀,卢晨曦,范玉雷,等.面向大数据流的分布式索引构建[J].软件学报,2021,32(11):3576-3595.

[36] 宋泊东,张立臣,江其洲.基于Spark的分布式大数据分析算法研究[J].计算机应用与软件,2019,36(01):39-44.

[37] Logothetis D, Olston C, Reed B, et al. Stateful bulk processing for incremental analytics[C]//ACM Symposium on Cloud Computing, SOCC 2010, In

中国知网 https://www.cnki.net

dianapolis, Indiana, USA, June. DBLP,2010:51-62.

[38] 魏森,周浩然,胡创等.基于混合内存的Apache Spark缓存系统实现与优化[J].计算机科学,2023,50(6):10-21.

[39] 宋宝燕,王俊陆,王妍.基于范德蒙码的HDFS优化存储策略研究[J].计算机学报,2015,38(9):1825-1837.

[40] 钱文渊,荆一楠,王晓阳,等.面向多表连接查询优化的基数估计方法[J].计算机工程,2022,48(06):167-173.

[41] 田杨,晏海华.一种基于自动分区的海量科学数据计算框架[J].北京航空航天大学学报,2022,48(06):1004-1012.

[42] Fararni A K, Nafis F, Aghoutane B , et al. Hybrid Recommender System for Tourism Based on Big Data and AI:A Conceptual Framework[J].Big Data Mining and Analytics,2021,4(01):47-55.

[43] Imran S ,Mahmood T ,Morshed A , et al. Big Data Analytics in Healthcare——A Systematic Literature Review and Roadmap for Practical Implementation[J].IEEE/CAA Journal of Automatica Sinica,2021,8(01):1-22.

[44] Isard M, Budiu M, Yu Y, et al. Dryad:distributed data-parallel programs from sequential building blocks[J]. Acm Sigops Operating Systems Review, 2007, 41(3):59-72.

[45] Munshi A A, Mohamed Y a-R I. Data Lake Lambda Architecture forSmart Grids Big Data Analytics [J]. IEEE Access, 2018, 6: 40463-40471.

[46] Ren Peng, Mao Ziyun, Li Shuaibo, et al. Intelligent Visualization System for Big Multi-source Medical Data Based on Data Lake [C]// Web Information Systems and Applications. Berlin: Springer, 2021: 706-717.

[47] Gufler B, Augsten N, Reiser A, et al. Load balancing in MapReduce based on scalable cardinality estimates[C]//The 28th IEEE International Conference on Data Engineering (ICDE). 2012: 522-533.

[48] Tang Z, Zhang X S, Li K, et al. An intermediate data placement algorithm for load balancing in Spark computing environment[J].Future Generation Computer Systems,2018,78:287-301.

[49] 郝晋瑶,牛保宁,康家兴.大规模路网图下关键词覆盖最优路径查询优化[J].软件学报,2020,31(08):2543-2556.

[50] 季航旭,姜苏,赵宇海等.分布式环境中的多作业执行调度策略与优化[J].计算机工程与科学,2021,43(06):951-961.

[51] 曾飞,杨雄,苏伟,等.基于区块链与数据湖的电力数据存储与共享方法[J].电力

中国知网 https://www.cnki.net

工程技术,2022,41(03):48-54.

[52] N. P S ,Jérôme D. DLBench+: A benchmark for quantitative and qualitative data lake assessment[J].Data Knowledge Engineering,2023,145:102154-102173.

[53] A. A S ,E. Y G ,S. A Z , et al. The concept of an intelligent data lake management system: machine consciousness and a universal data model[J]. Procedia Computer Science,2022,213407-414.

[54] Wu J ,Xiong J ,Dai H , et al. MIX-RS:A Multi-Indexing System Based on HDFS for Remote Sensing Data Storage[J].Tsinghua Science and Technology,2022,27(06):881-893.

[55] Athira N ,Divyansh M .An Overview of Data Warehouse and Data Lake in Modern Enterprise Data Management[J].Big Data and Cognitive Computing,2022,6(4):132-132.

[56] 阎逸飞. 基于 Key-Value 倾斜模型的 Spark 动态数据分区算法[D]. 北京:北京邮电大学,2019.

[57] 桂智明,孙忠祥,黄舟.区块链与数据库混合架构下的空间查询优化方法[J].北京大学学报(自然科学版),2023,59(02):261-270.

[58] 沈斯杰,陈榕,陈海波,等.基于图结构索引的分布式OLAP加速方法[J].软件学报,2023,34(10):4661-4680.

[59] 冯禹洪,吴坤汉,黄志鸿,等.基于FP-tree和MapReduce的集合相似度自连接算法[J].计算机研究与发展,2023,60(12):2890-2906.

[60] 黄梦霖. 面向 Spark 数据混洗中数据倾斜的动态均衡分区方法的研究[D]. 武汉:华中科技大学,2019.

[61] 李广龙,申德荣,聂铁铮,等.数据库外基于多模型的学习式查询优化方法[J].浙江大学学报(工学版),2022,56(02):288-296.

[62] 高旭东,王继选,高艳丰,等.Underwater pulsed spark discharge influenced by the relative position between the top of a pin electrode and an insulating tube[J].Plasma Science and Technology,2020,22(05):37-45.

[63] Sagi O, Rokach L. Approximating XGBoost with an interpretable decision tree[J]. Information Sciences, 2021, 572(2-3):522-542.

[64] Qu Y, Wu H, Xie P, et al. Carbon nanotube-carbon black/CaCu_3Ti_4O_(12) ternary meta composites with tunable negative permittivity and thermal conductivity fabricated by spark plasma sintering[J].Rare Metals,2023,42(12):4201-4211.

中国知网 https://www.cnki.net

[65] Zhang D ,Hao X ,Jia B , et al. Influences of oxide content and sintering temperature on microstructures and mechanical properties of intragranular-oxide strengthened iron alloys prepared by spark plasma sintering[J].International Journal of Minerals, Metallurgy and Materials,2023,30(09):1748-1755.

[66] 赵丽梅,黄小菊,宫学庆.Spark查询引擎中Join操作的优化[J].计算机应用与软件,2022,39(08):44-50.

[67] 刘晓宇,夏立斌,姜晓巍,等.HDFS分级存储系统元数据管理方法的研究[J].计算机工程与应用,2023,59(17):257-265.

[68] Rebecca E ,Corinna G ,Christoph G , et al. Modeling metadata in data lakes—A generic model[J].Data Knowledge Engineering,2021,136: 101931-1019318.

[69] Zhuo C ,Florian L V ,Heinz P , et al. Prediction of Sooting Index of Fuel Compounds for Spark-Ignition Engine Applications Based on a Machine Learning Approach[J].Journal of Thermal Science,2023,32(02):521-530.

[70] 李俊丽.基于Spark的倾斜数据虚拟划分算法[J].计算机工程与设计,2021,42(08):2271-2276.

[71] 廖湖声,黄珊珊,徐俊刚等.Spark性能优化技术研究综述[J].计算机科学,2018,45(07):7-15+37.

[72] 谭旭杰,邓长寿,董小刚等.SparkDE:一种基于RDD云计算模型的并行差分进化算法[J].计算机科学,2016,43(09):116-119+139.

[73] 夏立斌,刘晓宇,姜晓巍等.基于分布式数据集的并行计算框架内存优化方法[J].计算机工程,2023,49(04):43-51.

[74] 杨良怀,项俊腱,徐卫等.一种大数据流内存B+树构建方法[J].计算机科学,2018,45(03):173-179+214.

[75] Chambers B, Zaharia M. Spark: The definitive guide: Big data processing made simple[M]. California: O'Reilly Media, 2018:102-389.

[76] Karau H, Warren R. High performance Spark: best practices for scaling and optimizing Apache Spark[M]. California: O'Reilly Media, 2017:32-156.

[77] 罗永刚,陈兴蜀,杨露.基于内存优化配置的MapReduce性能调优[J].华南理工大学学报(自然科学版),2017,45(01):102-111.

[78] 张元鸣,虞家睿,陆佳炜等.基于Spark Streaming的视频大数据并行处理方法[J].计算机工程与科学,2021,43(10):1736-1743.

中国知网 https://www.cnki.net